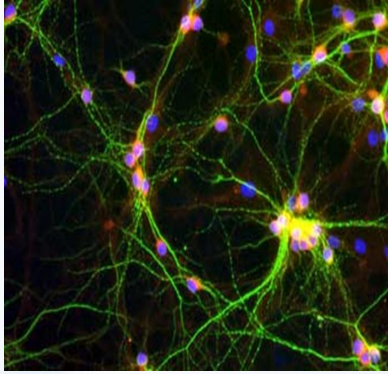
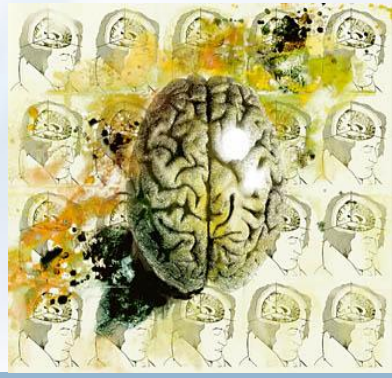
BIT.MLMR

Artificial Neural Networks

Lecture 3





Plutarch: the brain is not a container needs to be filled with knowledge, but a torch needs to be lighted





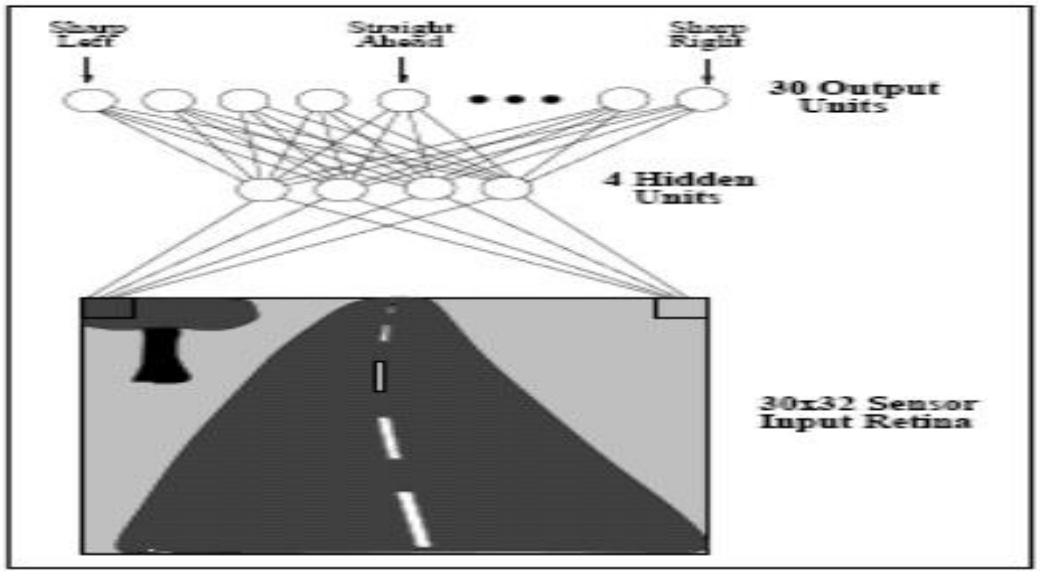


Google AutoCar

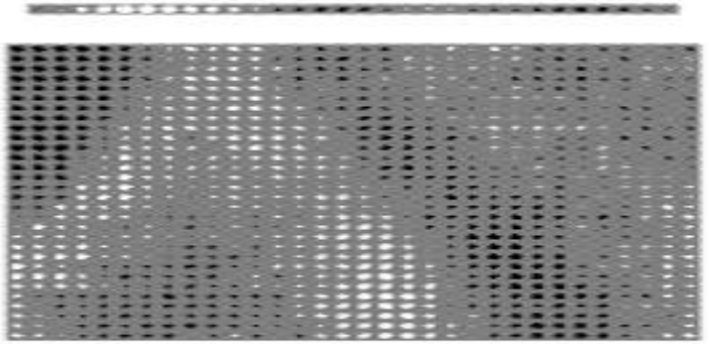


Neural network learning to steer an autonomous vehicle





Weight values for a hidden unit:



From T. M. Mitchell, *Machine Learning, 2006*

Neural Arts



Are these pictures created by ANN? If so, how does the ANN make it?

Outline

- What is Artificial Neural Networks (ANN)?
  - ANN Architectures
  - Learning approaches to ANN
- Multi-layer Perceptrons (MLP)
  - Perceptron
  - Back-Propagation (B-P) Algorithm and Network
  - Deep Learning
- Recurrent Network
  - Hopfield Network
  - Long Short-Term Memory (LSTM)
- Self-Organizing Feature Map (SOFM)

Part Ⅰ : What is ANN?

1.1 ANN

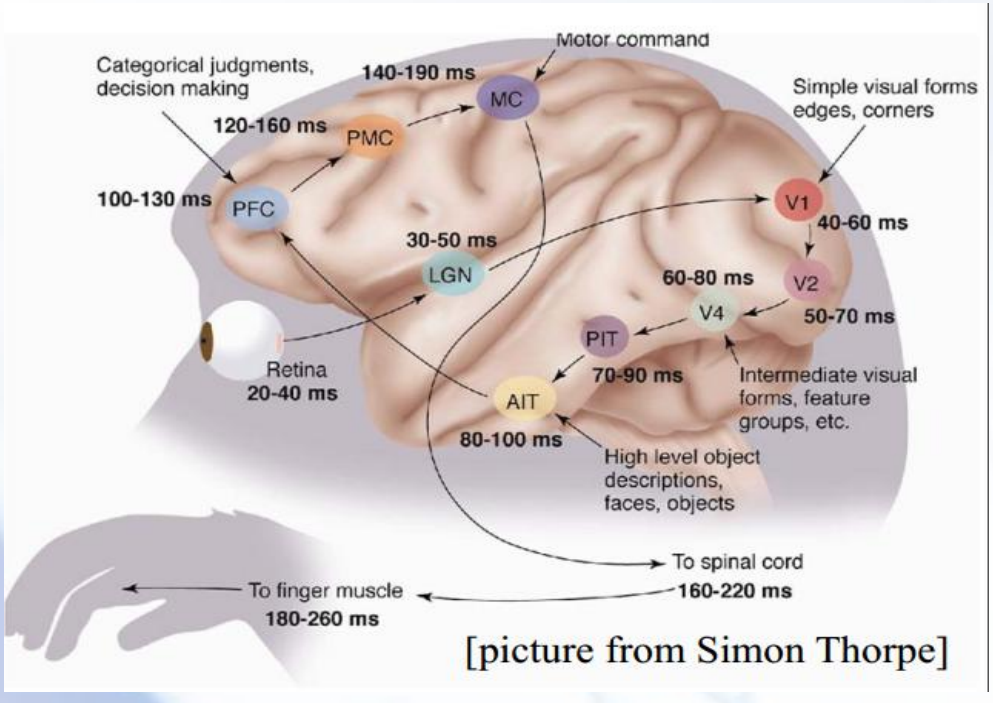
- **Individual units are interconnected that form a graph with many types of structures - acyclic or cyclic, directed or undirected.**

  Connectionism

- **Bottom-Up AI**

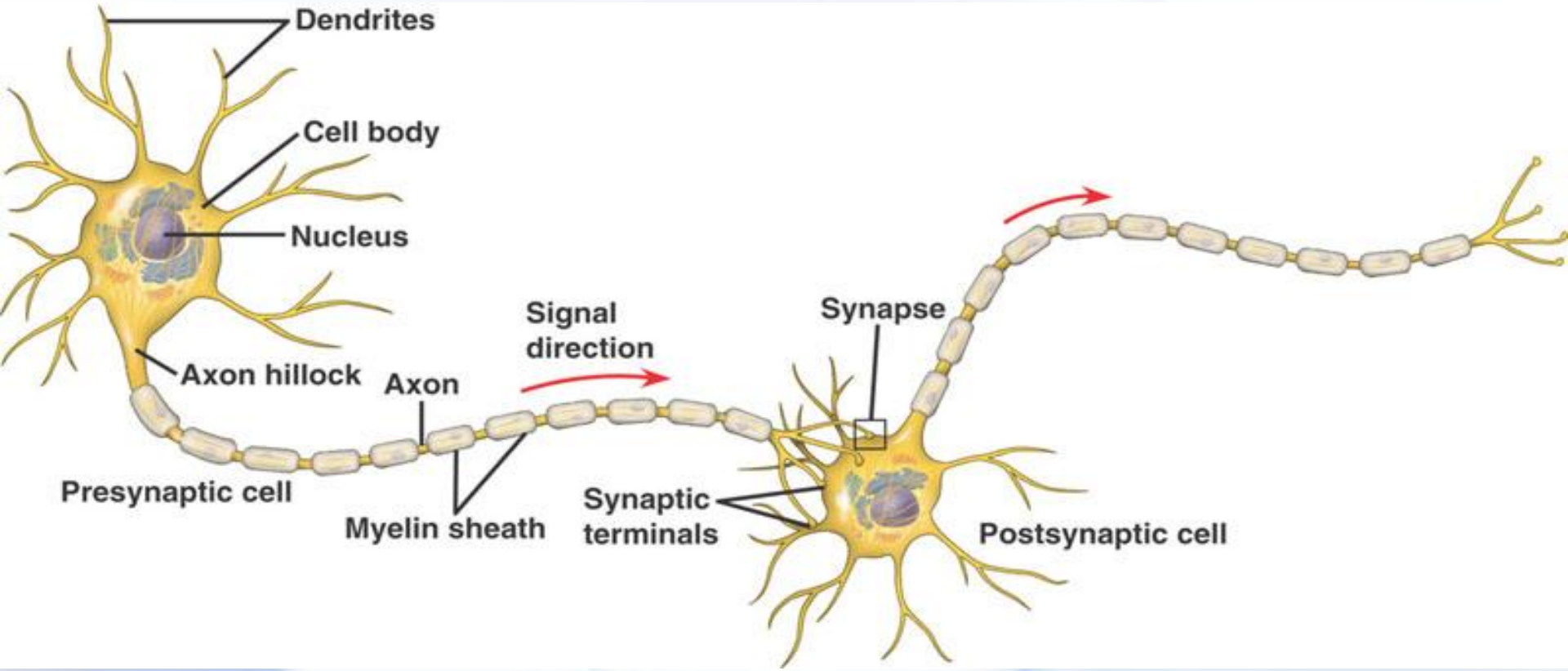
  Motivated by biological neural systems from simulation of structure to simulation of functionality

The brain


Motor command
Categorical judgments, decision making
140-190 ms
MC
Simple visual forms edges, corners
120-160 ms
PMC
100-130 ms
PFC
V1
40-60 ms
30-50 ms
LGN
60-80 ms
V2
V4
50-70 ms
PIT
Retina
20-40 ms
70-90 ms
Intermediate visual forms, feature groups, etc.
AIT
80-100 ms
High level object descriptions, faces, objects
To spinal cord
160-220 ms
To finger muscle
180-260 ms
[picture from Simon Thorpe]


SCIENCEphotoLIBRARY

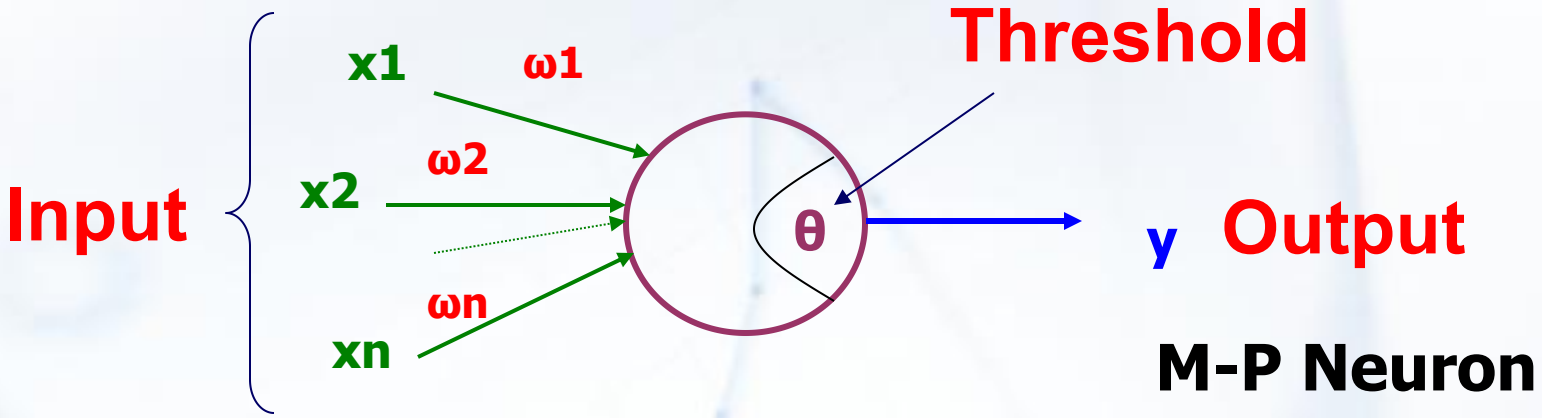
Biological Neurons



- Temporal and Spatial Summation
- Excitation and Inhibition

Picture：http://kvhs.nbed.nb.ca/gallant/biology/neuron_structure.jpg

Artificial Neurons



$$y = f(\xi) = f(g(X))$$

***f*: Activation Function**

g: Combination Function

McClloch and Pitts, *A logical calculus of the ideas immanent in nervous activity*, 1943

Combination Functions

❖ Weighted Sum

$$\xi = g(\boldsymbol{X}) = \sum_{i=1}^{n} \omega_i x_i - \theta$$

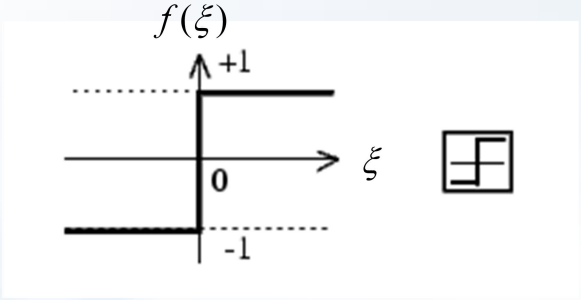
$$y = f(\xi) = f\left(\sum_{i=1}^{n} \omega_i x_i - \theta\right) \longrightarrow y = f(\xi) = f\left(\sum_{i=0}^{n} \omega_i x_i\right)$$

❖ Radial Distance

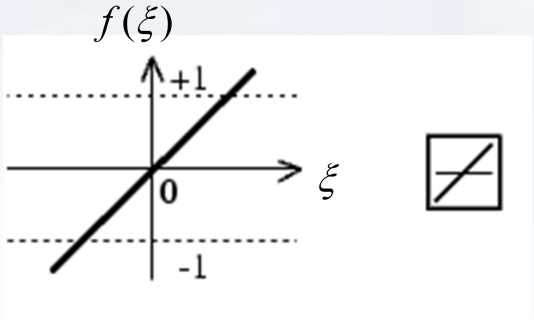
$$\xi = \|\boldsymbol{X} - \boldsymbol{C}\| = \sqrt{\sum_{i=1}^{n} (x_i - c_i)^2}$$

$$y = f(\xi) = f\left(\sqrt{\sum_{i=1}^{n} (x_i - c)^2}\right)$$

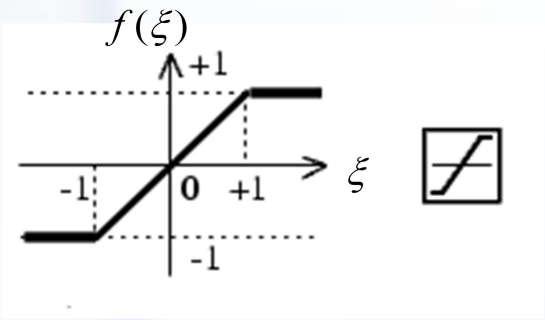
Activation Functions



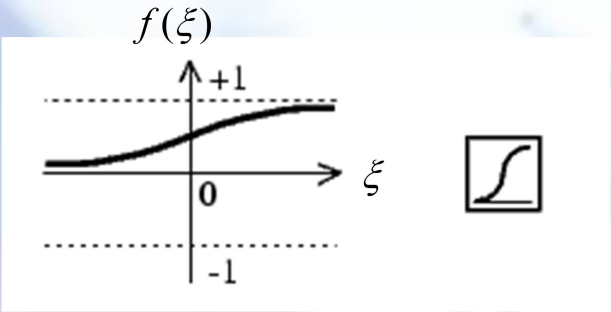
Threshold



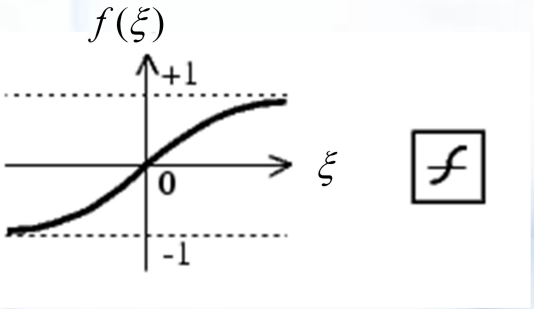
Linear



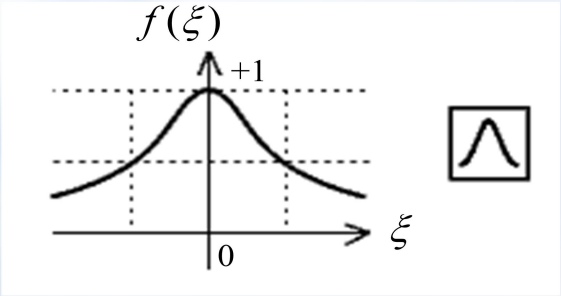
Saturating Linear



Logistic Sigmoid



Hyperbolic tangent Sigmoid



Gaussian

Two main problems in ANN

- **Architectures**

  How to interconnect individual units?

- **Learning Approaches**

  How to automatically determine the connection weights or even structure of ANN?

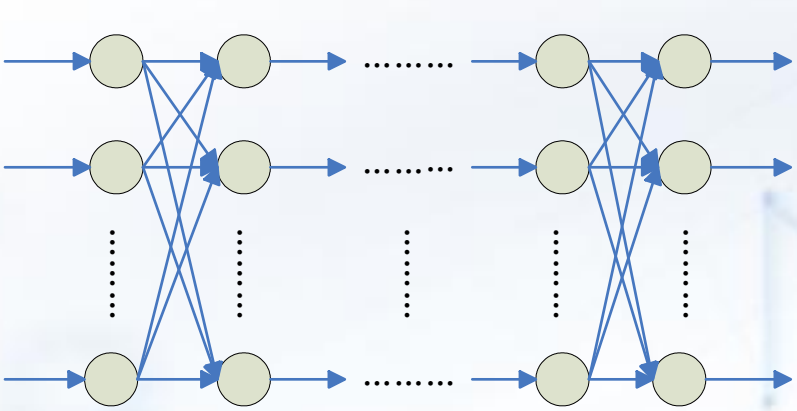
Solutions to these two problems leads to a concrete ANN!

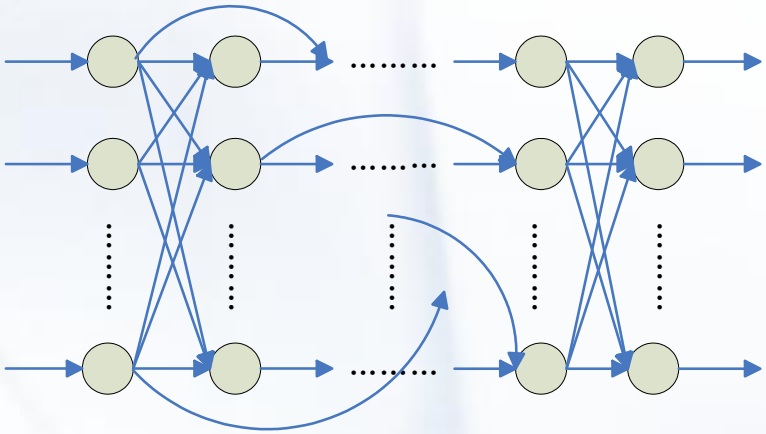
1.2 ANN Architectures

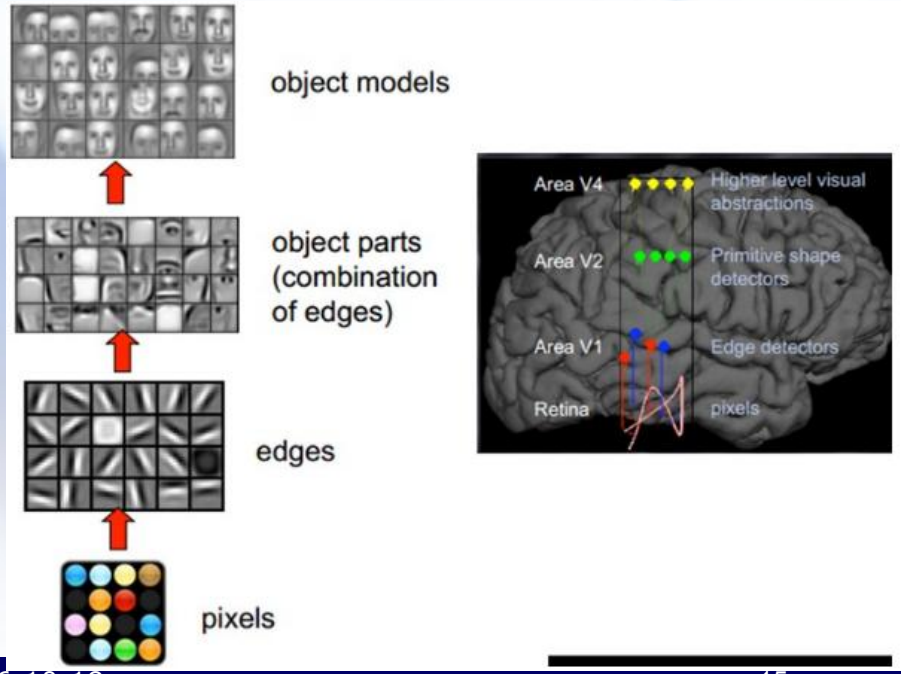
- **Feedforward Architecture**
  - **without loops**
  - **static**

- **Feedback (Recurrent) Architecture**
  - **with loops**
  - **dynamic (non-linear dynamical systems)**

Feedforward networks

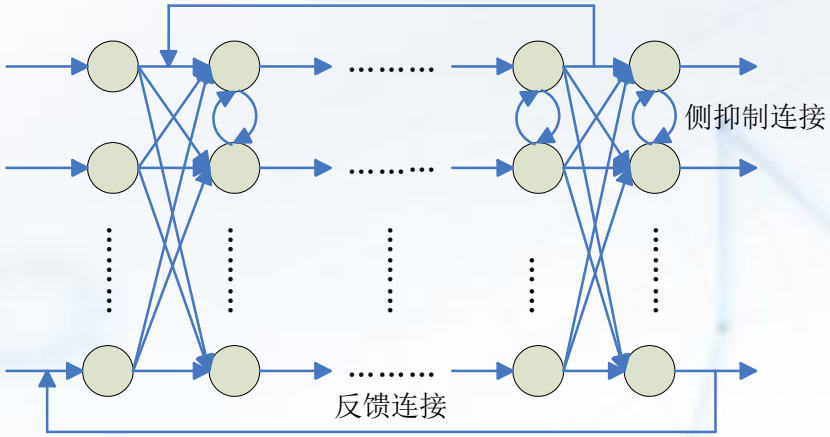


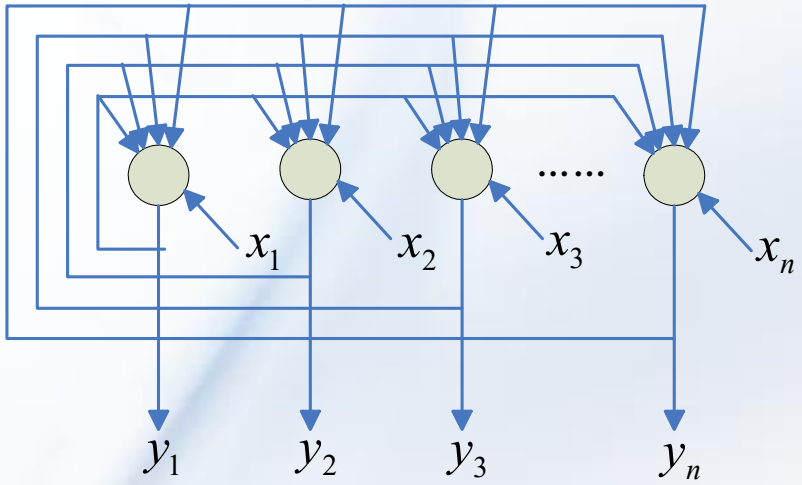




signal→abstract、
iteration→sophistic
ated abstract

Feedback Architectures





1.3 Learning Approaches to ANN

- A process by which the free parameters of a neural network are adapted through a process of stimulation by the environment in which the network is embedded.
- Learning Model

  Incremental vs. Batch
- Two categories

  Supervised vs. Unsupervised

Learning Strategies: Error Correction

- **Minimize the error between the actual outputs and expected outputs for inputs (Supervised)**
  - **Delta Rule (LMS Rule, Widrow-Hoff)**
  - **B-P Learning**

Objective: $$\omega^* = \arg\min_{\omega} \frac{1}{K} \sum_{k=1}^{K} e(\boldsymbol{D}_k, \boldsymbol{Y}_k)$$

Solution: $$\omega \leftarrow \omega + \Delta\omega = \omega + \eta\delta$$

Learning Strategies: Hebbrian Learning

- Strengthening connection weights when both endpoints activated
- Unsupervised Learning

$$\omega_{ij}(t+1) = \omega_{ij}(t) + \eta\left(x_i(t) \cdot x_j(t)\right)$$

Learning Strategies: Competitive Learning

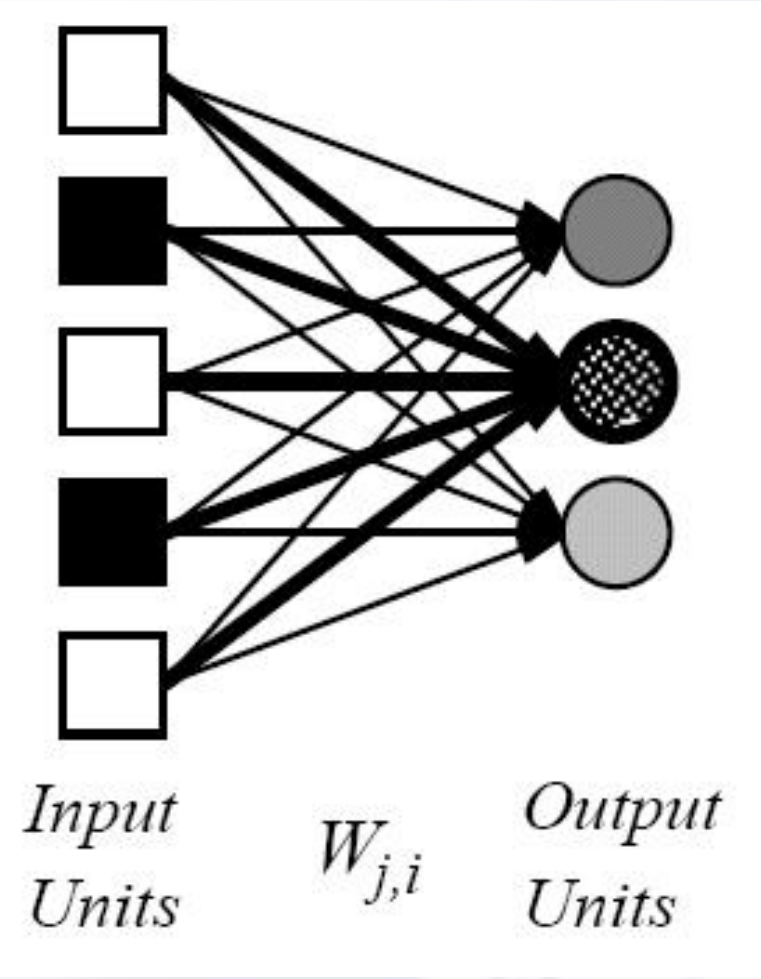
- Winner-take-all (Unsupervised)
- How to compete?
  - Hard competition

    Only one neuron is activated
  - Soft competition

    Neurons neighboring the true winner are activated.

Part Ⅱ: Multi-layer Perceptrons (MLP)

2.1 Single-layer Perceptron



- **Binary Threshold Activation Function**
- **Output units all operate separately, i.e. no shared weights**

Rosenblatt, 1957

Single-layer Perceptron Learning

- Adjust weights to reduce error on training set
- Simple weight update rule:

$$\omega_{ij} \leftarrow \omega_{ij} + \eta \underbrace{x_i \left(d_j - y_j \right)}_{\delta}$$

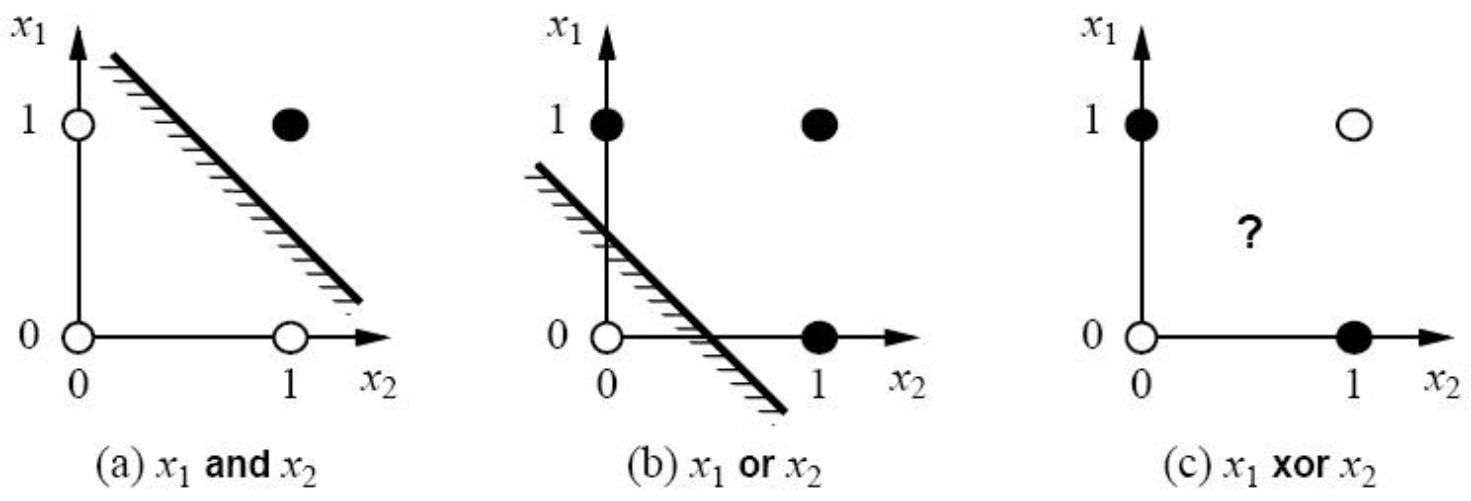
desired output

generated output

$$y_j = f\left(\sum_{i=0}^{m} \omega_{ij} x_i \right)$$

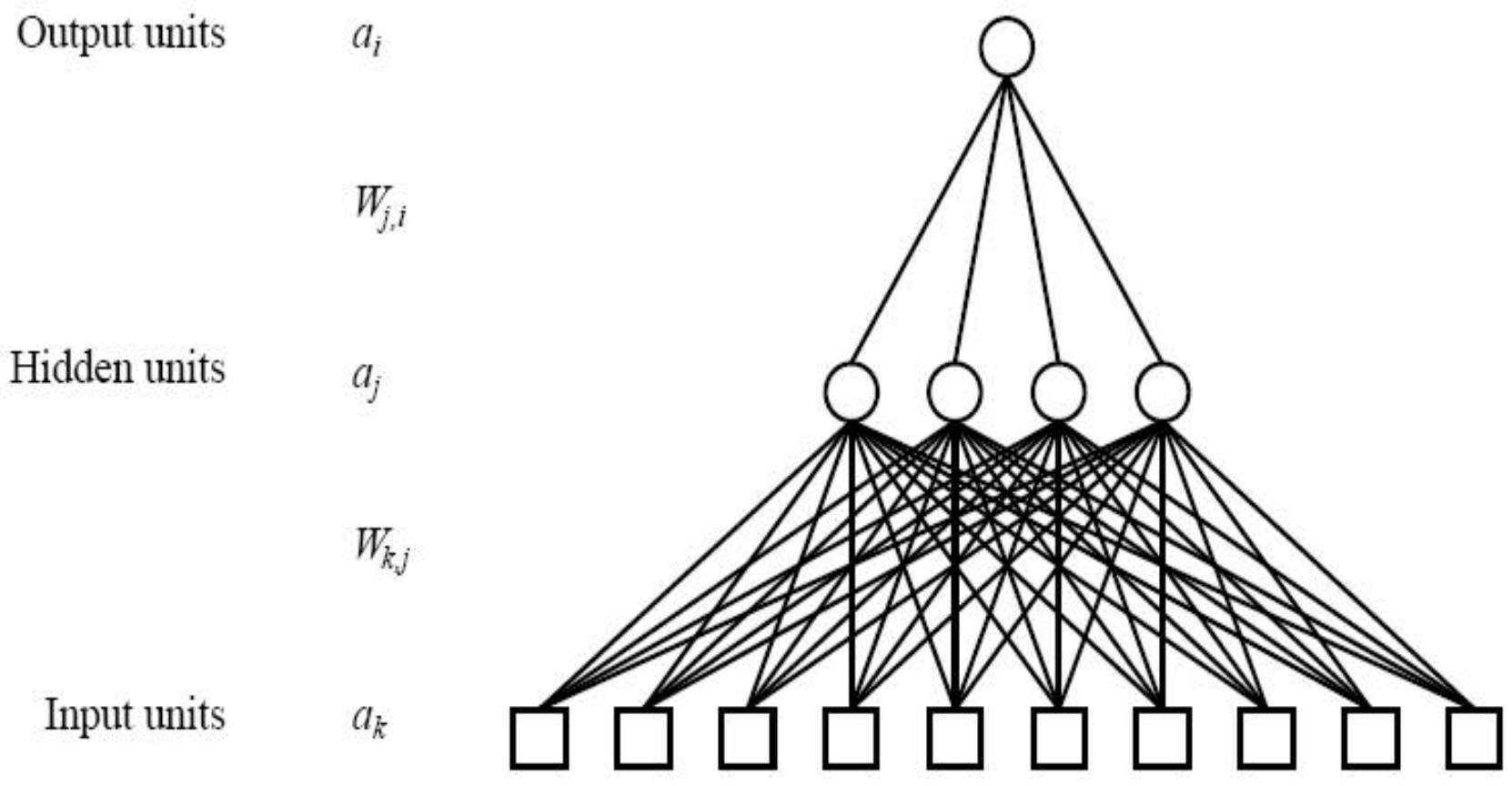
Expressiveness of Single-layer Perceptron

- Can represent AND, OR, NOT, etc., but not XOR
- Represents a linear separator :



Minsky & Papert (1969) pricked the neural network balloon

Multi-layer Perceptron (MLP)



Deep Architectures

Deep architectures are composed of multiple levels (usually >3) of non-linear operations, such as neural nets with many hidden layers.

Usefulness of Deep Architectures

- **Eg., learning feature hierarchies, where features from higher level of the hierarchy are formed by lower level features.**

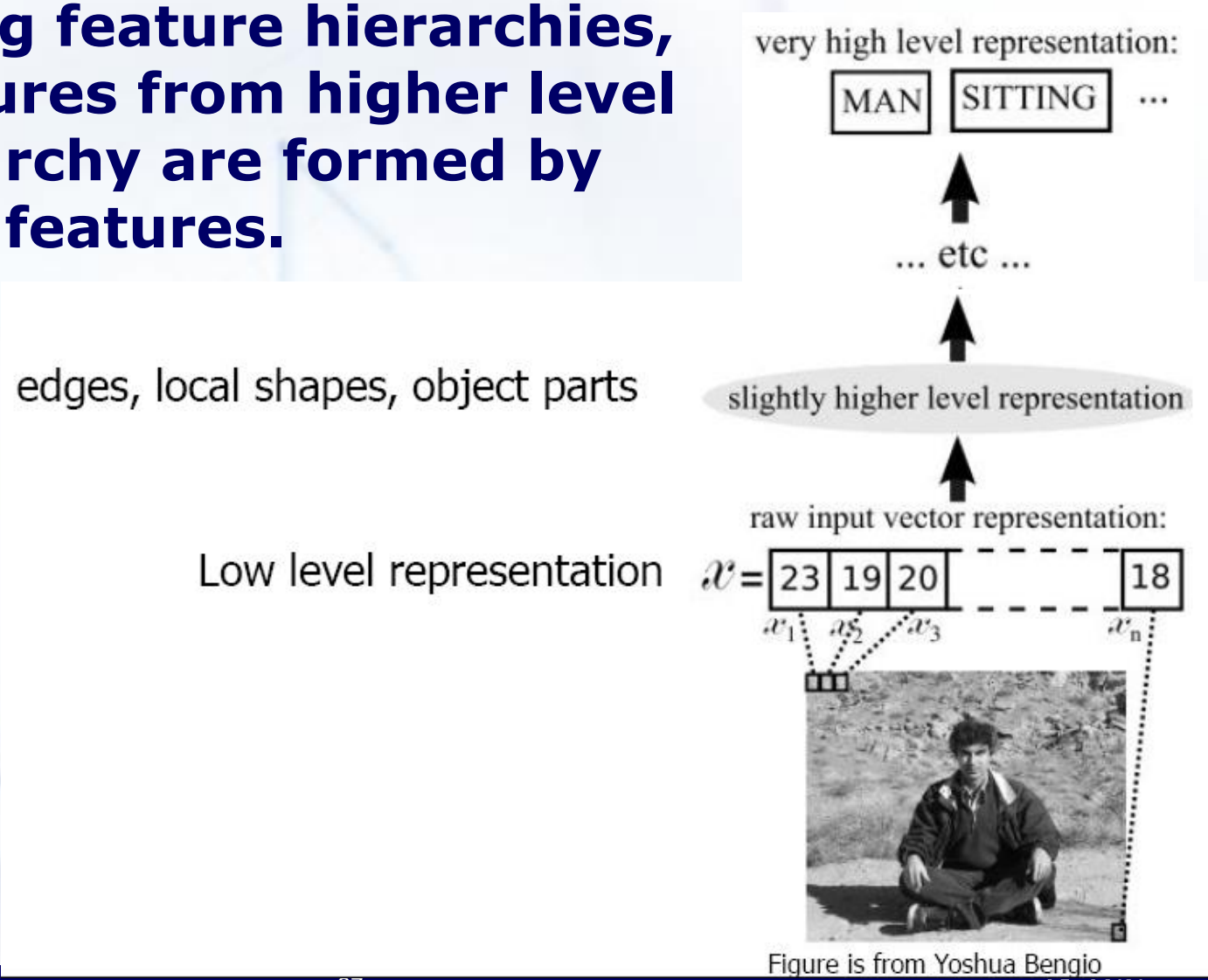


Figure is from Yoshua Bengio

Advantages of Deep Architectures

- **Neurobiological Motivation:** The mammal brain is organized in a deep architecture (Serre, Kreiman, Kouh, Cadieu, Knoblich, & Poggio, 2007) (E.g. visual system has 5 to 10 levels)
- Some functions cannot be efficiently represented (in terms of number of tunable elements) by architectures that are too shallow.
- Deep architectures might be able to represent some functions otherwise not efficiently representable.

Advantages of Deep Architectures Cont'd

- **More formally**: Functions that can be compactly represented by a depth k architecture might require an exponential number of computational elements to be represented by a depth k − 1 architecture
- The consequences are
- **Computational**: We don't need exponentially many elements in the layers
- **Statistical**: poor generalization may be expected when using an insufficiently deep architecture for representing some functions.

Expressiveness of MLPs

- **3 layers: All continuous functions**
- **4 layers: all functions**

How to learn the weights???

waiting B-P algorithm until 1986

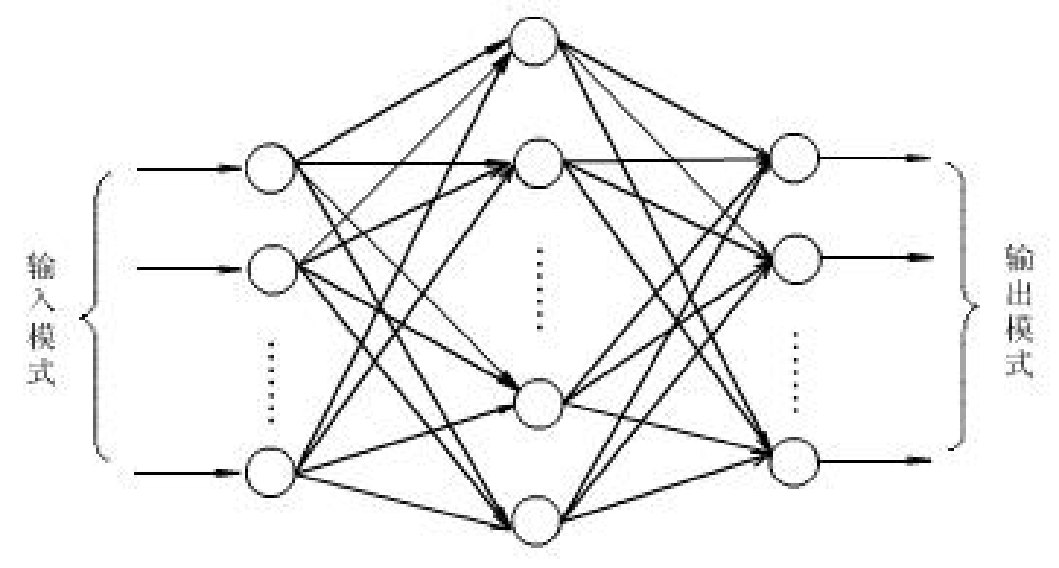
- **More Layers:**

waiting Deep Learning until 2006

2.2 B-P Network

- **Architecture**

  A kind of multi-layer perceptron, in which the Sigmoid activation function is used.

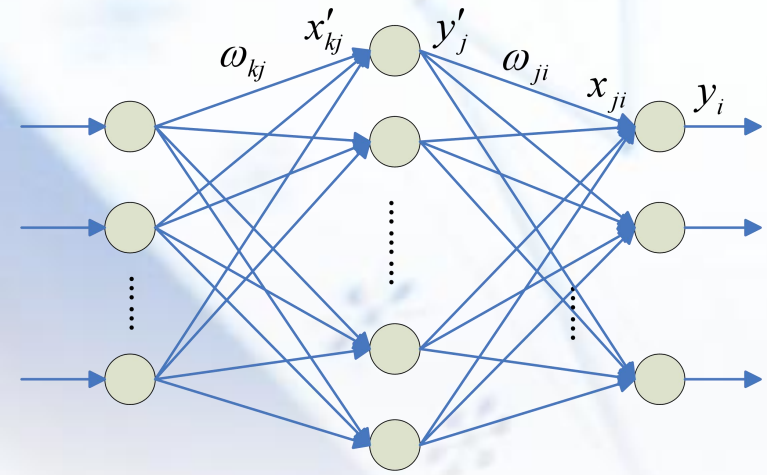


B-P Learning Algorithm

- **Learning approach**

  - **Input data** was **put forward** from input layer to hidden layer, then to out layer

  - **Error information** was **propagated backward** from out layer to hidder layer, then to input layer



Rumelhart & Meclelland, *Nature,1986*

B-P Learning Steps

Step1. Select a pattern from the training set and present it to the network.

Step2. Compute activation of input, hidden and output neurons in that sequence.

Step3. Compute the error over the output neurons by comparing the generated outputs with the desired outputs.

Step4. Use the calculated error to update all weights in the network, such that a global error measure gets reduced.

Step5. Repeat Step1 through Step4 until the global error falls below a predefined threshold.

B-P Derivation

❖ Global Error Measure

generated output

$$e(\omega) = \frac{1}{2}\sum_{i=1}^{n}[d_i - y_i]^2$$

desired output

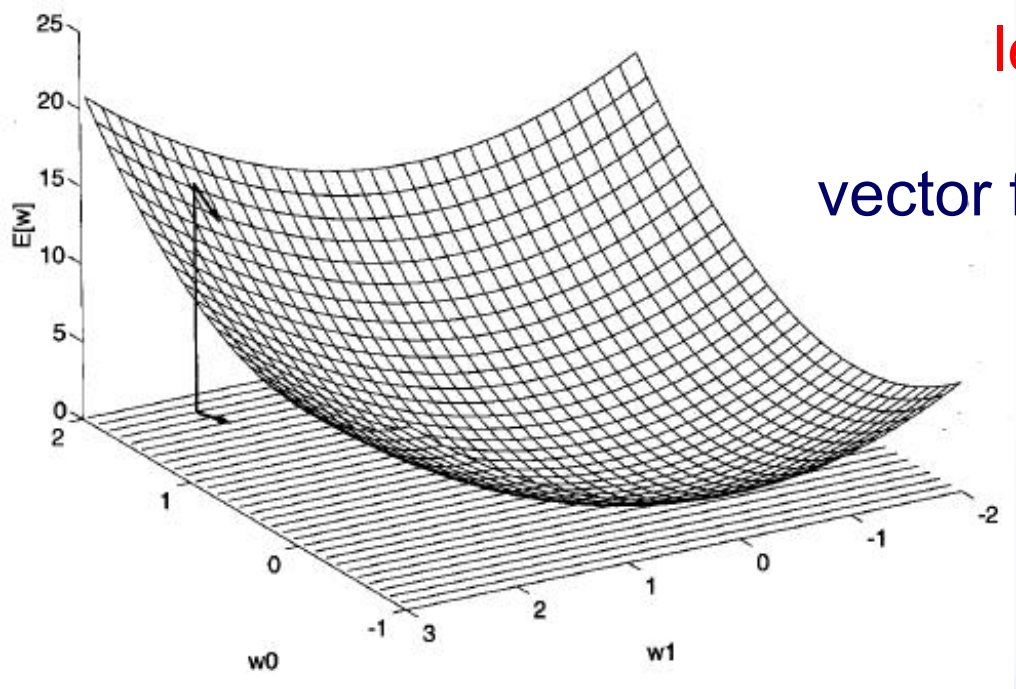
squared error

$$\omega^* = \arg\min_{\omega} e(\omega)$$

The objective is to minimize the squared error, i.e. reach the Minimum Squared Error (MSE)

Gradient Descent

- **Optimization method for finding out the weight vector leading to the MSE**



learning rate gradient

vector form:

$$\boldsymbol{\omega} = \boldsymbol{\omega} + \eta\underbrace{\left[-\nabla e(\boldsymbol{\omega})\right]}_{\delta}$$

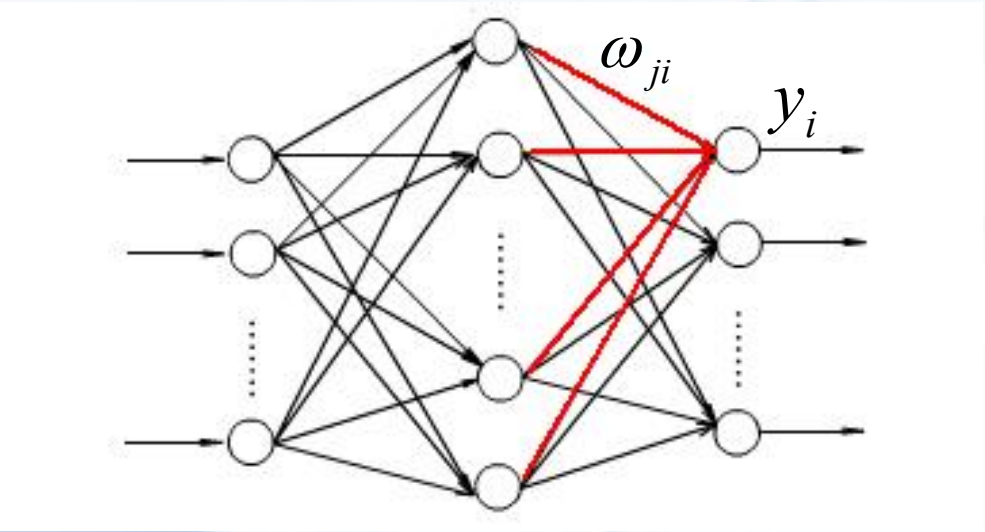
element form:

$$\omega = \omega - \eta\frac{\partial e}{\partial \omega}$$

Weight Update Rule

- **For output layer:**

$$\omega_{ji} = \omega_{ji} - \eta x_{ji} y_i (1 - y_i)(y_i - d_i)$$



$$\frac{\partial e}{\partial \omega_{ji}} = \frac{\partial e}{\partial y_i} \frac{\partial y_i}{\partial \sigma_i} \frac{\partial \sigma_i}{\partial \omega_{ji}}$$

Weight Update Rule

- **For output layer:**

$$\omega_{ji} = \omega_{ji} - \eta x_{ji} y_i (1 - y_i)(y_i - d_i)$$

$$e = \frac{1}{2}\sum (d_i - y_i)^2 \Rightarrow \frac{\partial e}{\partial y_i} = (y_i - d_i)$$

$$y_i = \frac{1}{1 + e^{-\sigma_i}} \Rightarrow \frac{\partial y_i}{\partial \sigma_i} = y_i (1 - y_i)$$

$$\sigma_i = \sum \omega_{ji} x_{ji} \Rightarrow \frac{\partial \sigma_i}{\partial \omega_{ji}} = x_{ji}$$

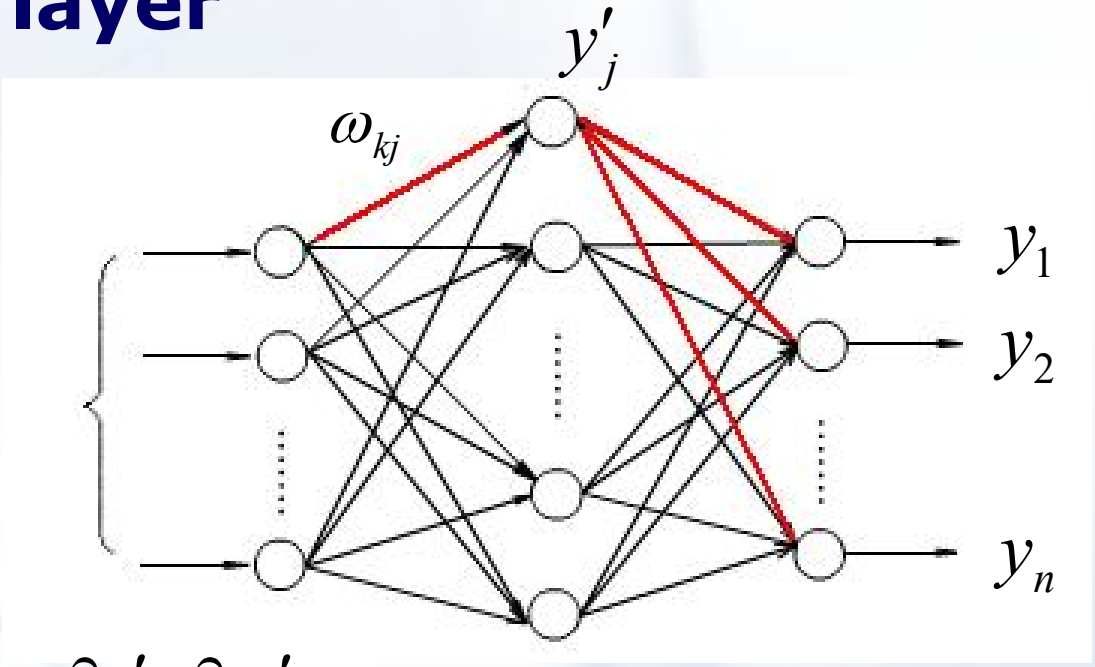
$$\frac{\partial e}{\partial \omega_{ji}} = x_{ji} y_i (1 - y_i)(y_i - d_i)$$

$$\Delta \omega_{ji} = -\eta \frac{\partial e}{\partial \omega_{ji}}$$

Weight Update Rule

- **For hidden layer**

$$e(\boldsymbol{\omega}) = \frac{1}{2}\sum_{i=1}^{n}[d_i - y_i]^2$$



$$\frac{\partial e}{\partial \omega_{kj}} = \sum_{i=1}^{n}\frac{\partial e}{\partial y_i}\frac{\partial y_i}{\partial \sigma}\frac{\partial \sigma}{\partial y'_j}\frac{\partial y'_j}{\partial \sigma'}\frac{\partial \sigma'_j}{\partial \omega_{kj}}$$

$$\omega_{ji}\, y_i(1-y_i)(y_i - d_i)$$

Weight Update Rule

- **For hidden layer**

$$\frac{\partial e}{\partial y_i} = (y_i - d_i), \quad \frac{\partial y_i}{\partial \sigma_i} = y_i(1 - y_i), \quad \frac{\partial \sigma_i}{\partial y'_j} = \omega_{ji}, \frac{\partial y'_j}{\partial \sigma'_j} = y'_j(1 - y'_j), \frac{\partial \sigma'_j}{\partial \omega_{kj}} = x_{kj}$$

$$\frac{\partial e}{\partial \omega_{kj}} = x_{kj} y'_j (1 - y'_j) \sum_{i=1}^{n} [\omega_{ji} y_i (1 - y_i)(y_i - d_i)] \qquad \Delta\omega_{kj} = -\eta \frac{\partial e}{\partial \omega_{kj}}$$

$$\omega_{kj} = \omega_{kj} - \eta x_{kj} y'_j (1 - y'_j) \sum_{i=1}^{n} \omega_{ji} y_i (1 - y_i)(y_i - d_i)$$

Application1：face detection

◆ **The component of vertical face recognition in RBK neural network for face detection**

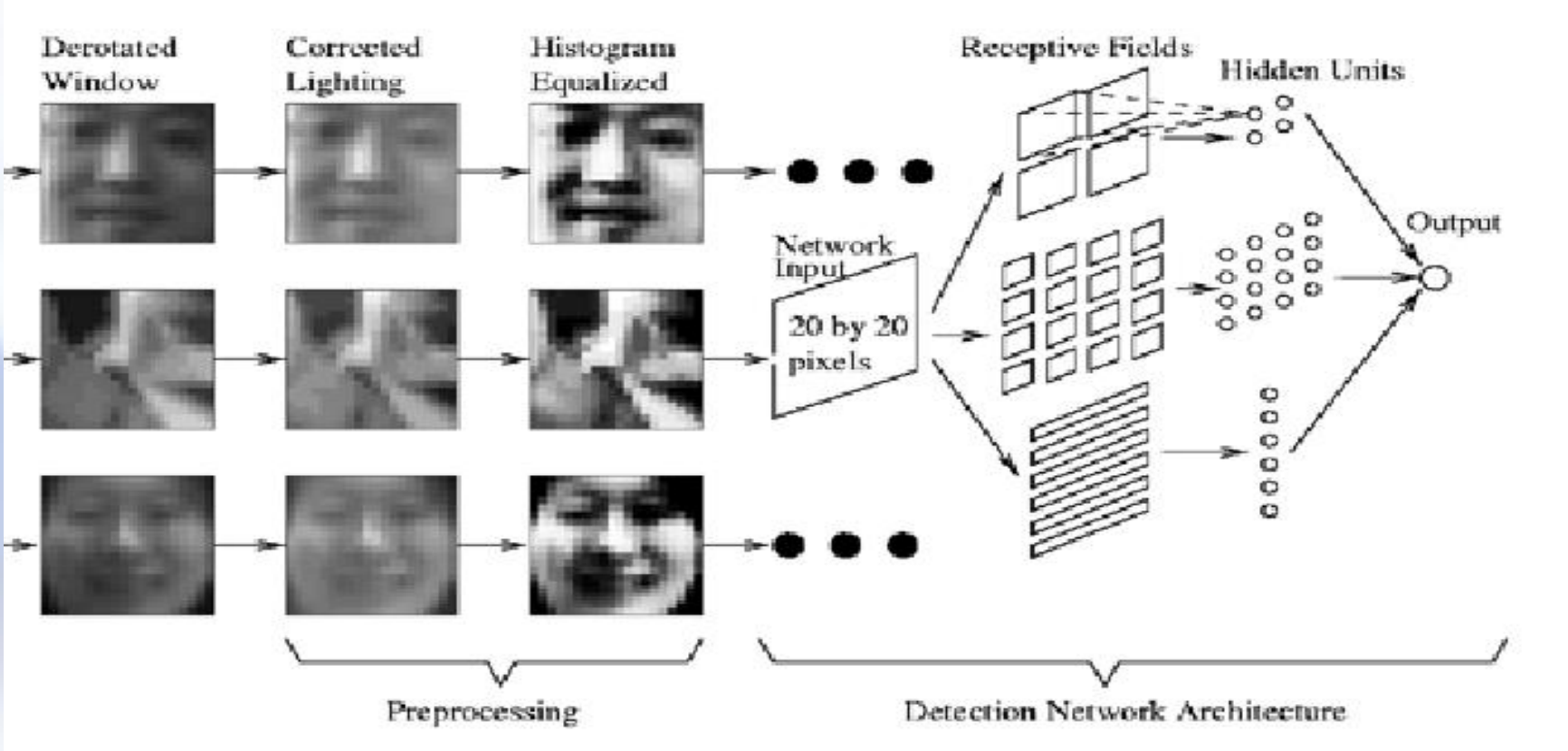
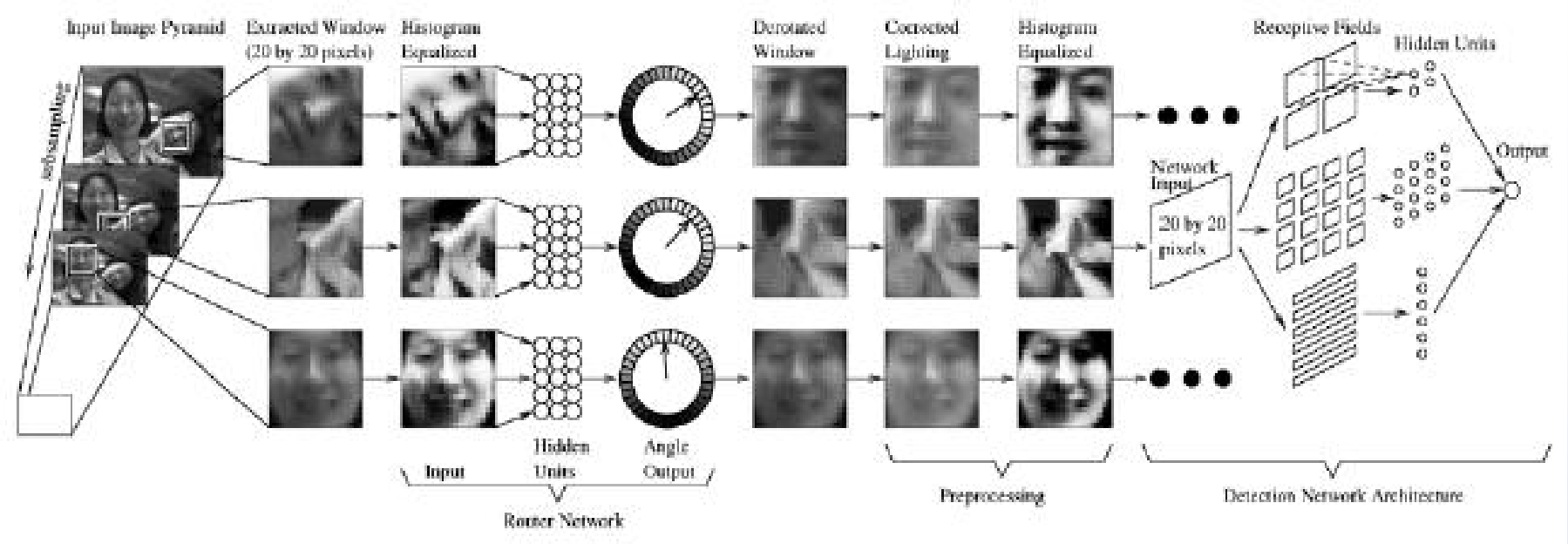


Figure from "Rotation invariant neural-network based face detection," H.A. Rowley, S. Baluja and T. Kanade, Proc. Computer Vision and Pattern Recognition, 1998, copyright 1998, IEEE

Application1：face detection

- **The complete RBK neural network for face detection**



It is composed of two neural networks. The first ANN is used to detect face directions in sub-windows. Then the sub-windows is rectified by using the detected directions to obtain vertical faces. Finally, the rectified sub-windows are put forward to the second ANN for face recognition.

Figure from "Rotation invariant neural-network based face detection," H.A. Rowley, S. Baluja and T. Kanade, Proc. Computer Vision and Pattern Recognition, 1998, copyright 1998, IEEE

Application1：face detection

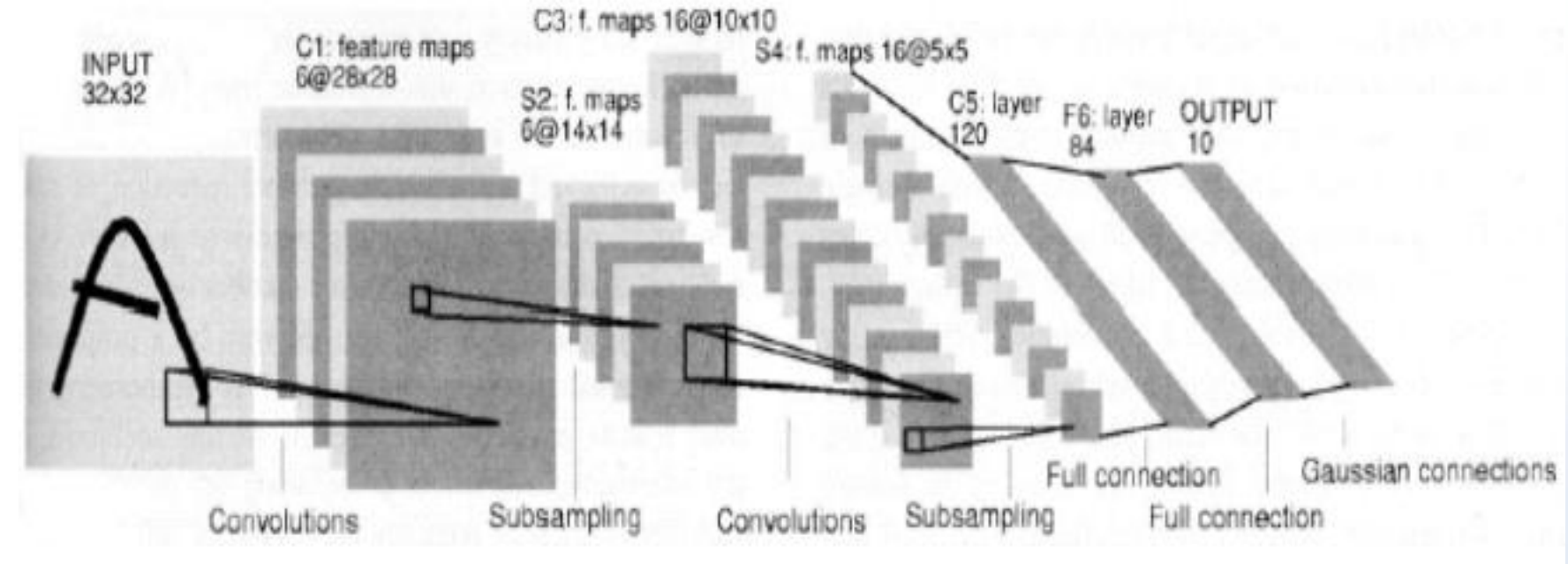


◆**The experimental results of RBK neural network for face detection**

Figure from “Rotation invariant neural-network based face detection,” H.A. Rowley, S. Baluja and T. Kanade, Proc. Computer Vision and Pattern Recognition, 1998, copyright 1998, IEEE

Application2：character recognition

- Convolutional neural network of Lecun, called LeNet-5 for short



It includes two groups of procedures of filtering and sub-sampling, which can be thought as a feature extraction process. The recognition is performed on the output of this process.

Figure from "Gradient-Based Learning Applied to Document Recognition", Y. Lecun et al Proc. IEEE, 1998 copyright 1998, IEEE

Application2：character recognition

◆LeNet-5 experimental results



Fig. 4. Size-normalized examples from the MNIST database.

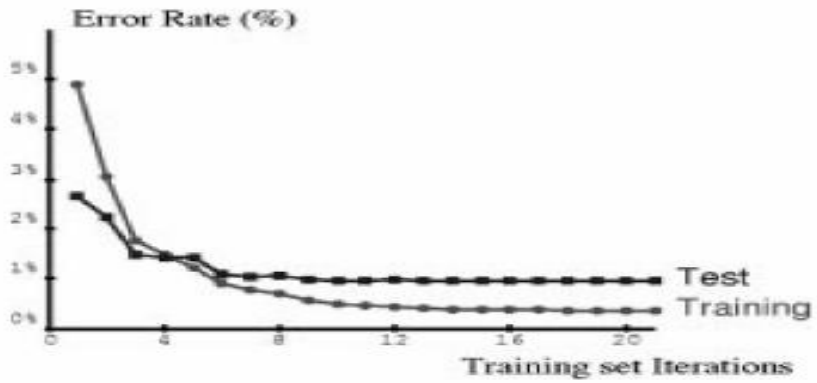


Fig. 5. Training and test error of LeNet-5 as a function of the number of passes through the 60 000 pattern training set (without distortions). The average training error is measured on-the-fly as training proceeds. This explains why the training error appears to be larger than the test error initially. Convergence is attained after 10–12 passes through the training set.

- ❖ 3-nearest-neighbor = 2.4% error
- ❖ 400×300×10 unit MLP = 1.6% error
- ❖ LeNet-5 = 0.9% error
- ❖ Current best (SVMs) ≈ 0.4% error

Figure from “Gradient-Based Learning Applied to Document Recognition”, Y. Lecun et al Proc. IEEE, 1998 copyright 1998, IEEE

Transformer

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

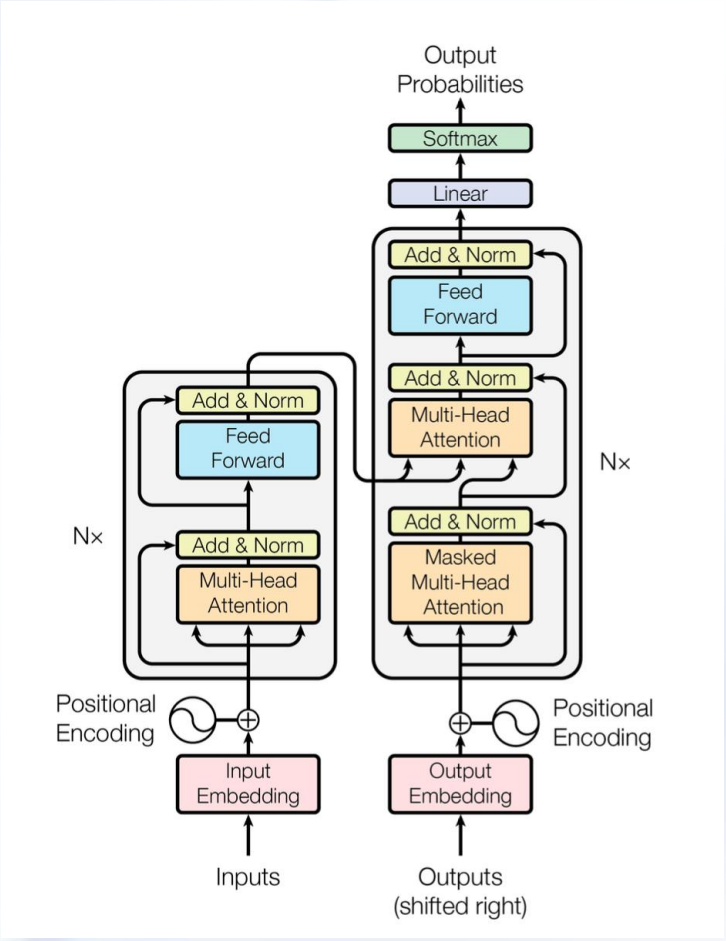
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

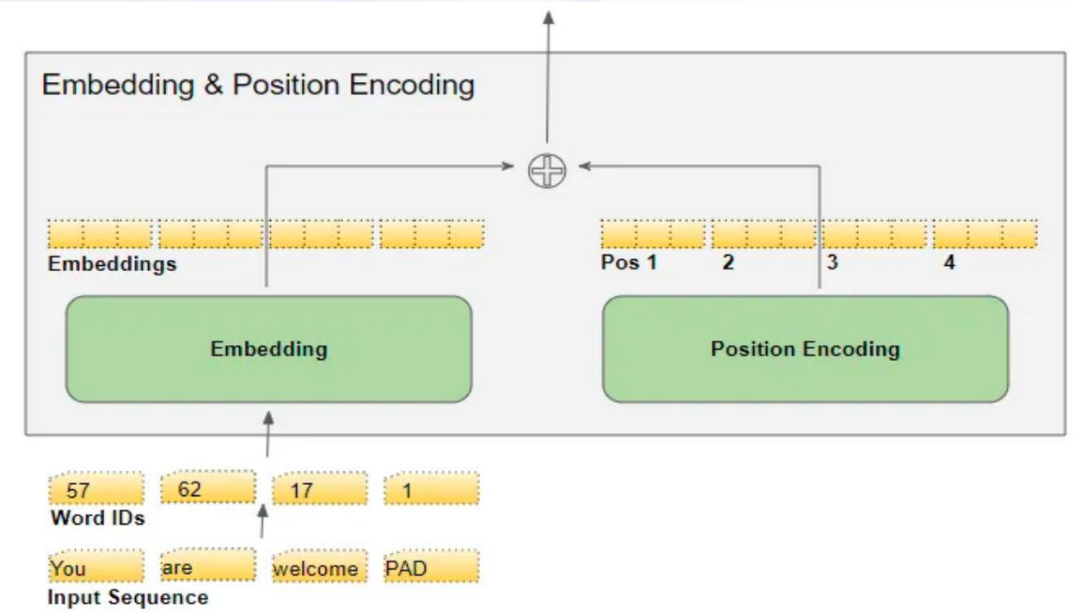
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.



Step 1： Input & Positional Encoding

· **Word Embedding:**
Maps discrete text tokens into continuous vectors of a fixed dimension.

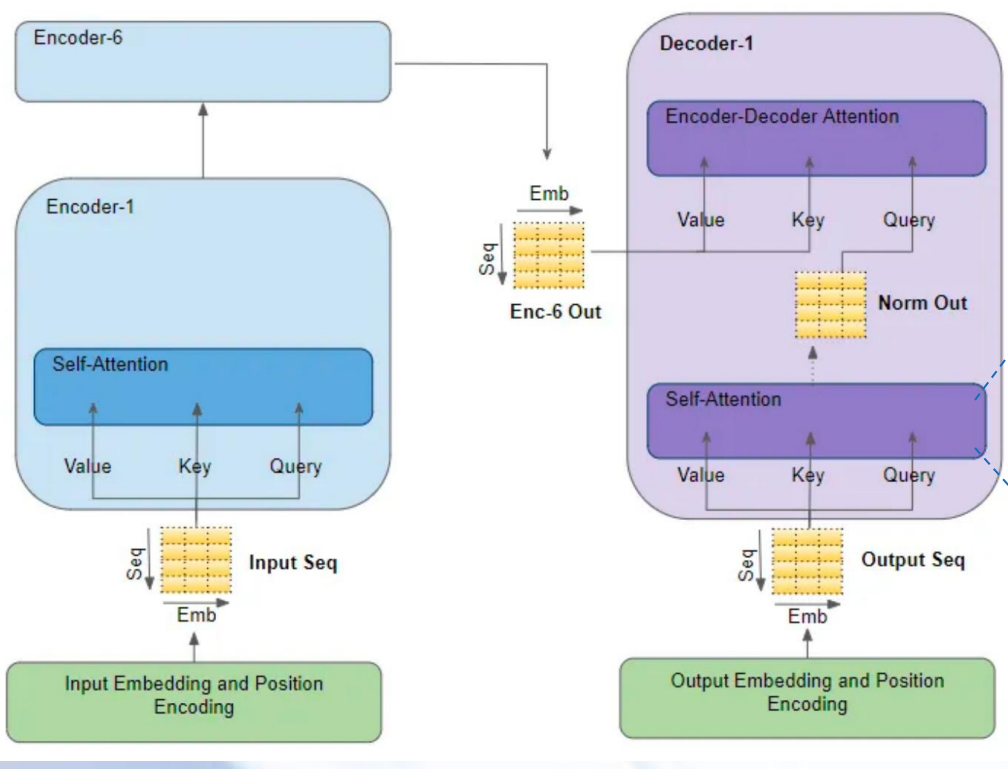
· **Positional Encoding:**
Transformers process all words in parallel (no RNN sequence).

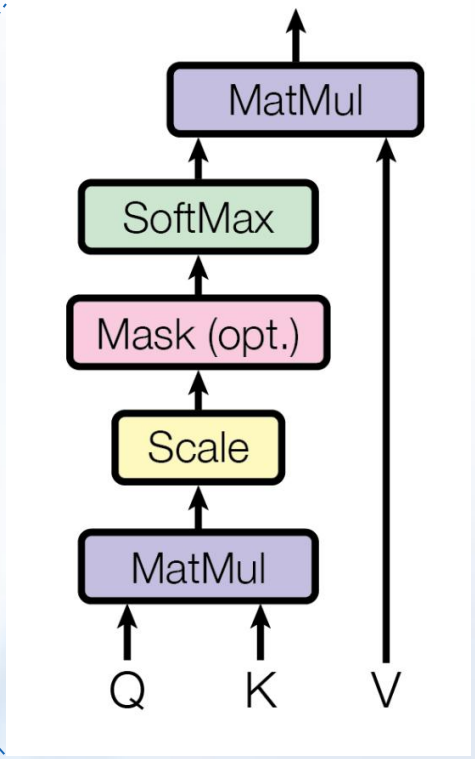


$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Step 2: Scaled Dot-Product Attention





$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$\boldsymbol{QK^T}$: Calculates similarity scores between words.

$\sqrt{\boldsymbol{d_k}}$: Scales down the scores to prevent gradient vanishing in the softmax function.

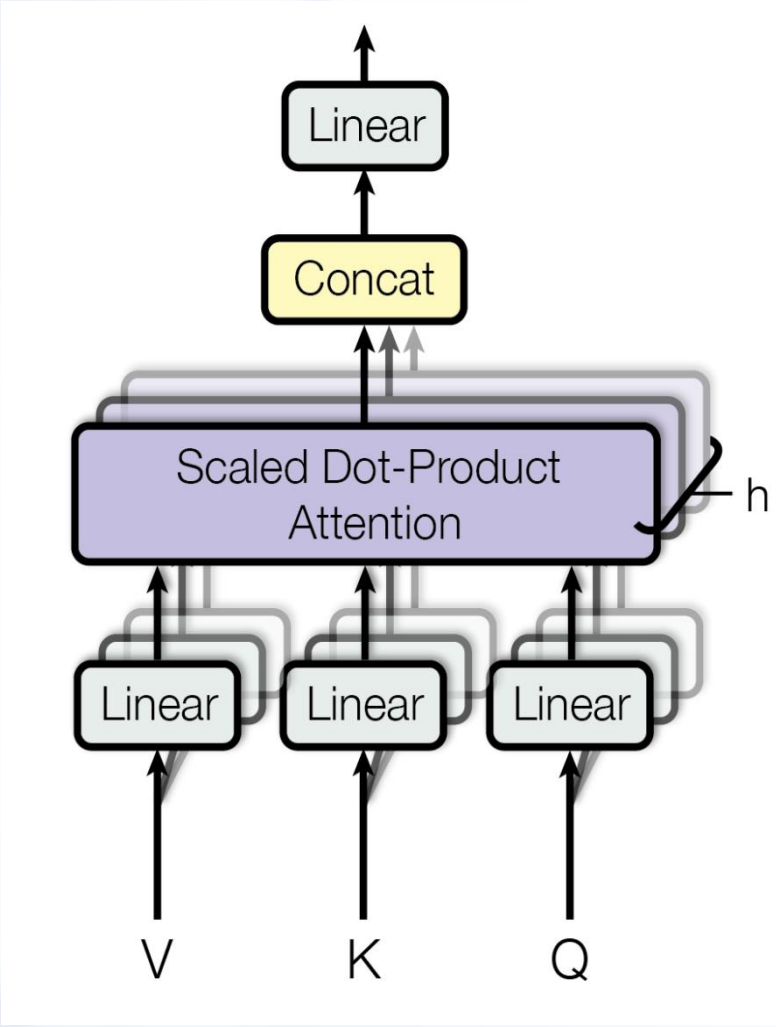
Step 3: Multi-Head Attention

Multiple "heads" allow the model to learn different contextual relationships in parallel subspaces.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Outputs: all heads concatenated and multiplied by a final weight matrix W^O



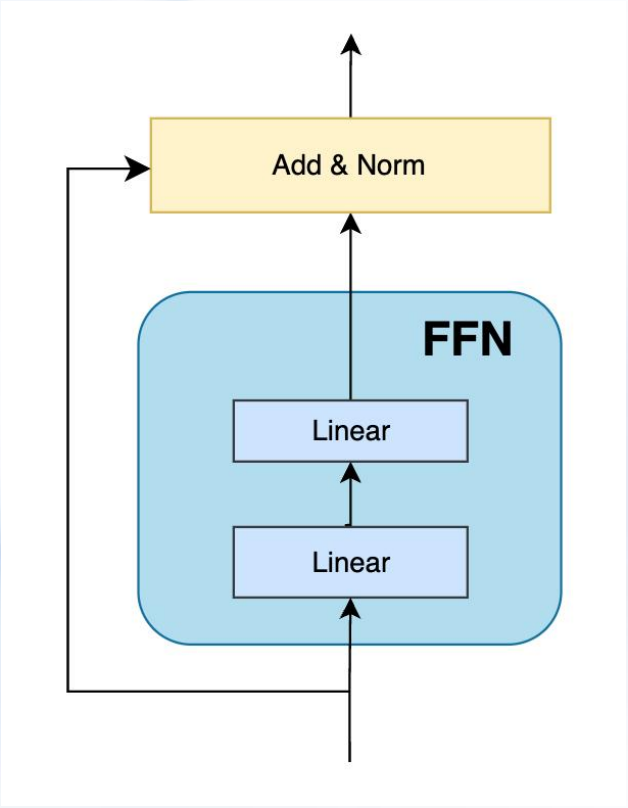
Step 4: FFN & Residual Connections

· **Position-wise Feed-Forward Network:** Adds non-linear processing power to each position independently.

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Add & Norm: Residual connections, followed by layer normalization.

$$\mathrm{Output} = \mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$$



DBN and Autoencoders

- **Deep Belief Networks (DBN)**

  Hinton, G. E, Osindero, S., and Teh, Y. W. (2006). A fast learning algorithm for deep belief nets. Neural Computation, 18:1527-1554.

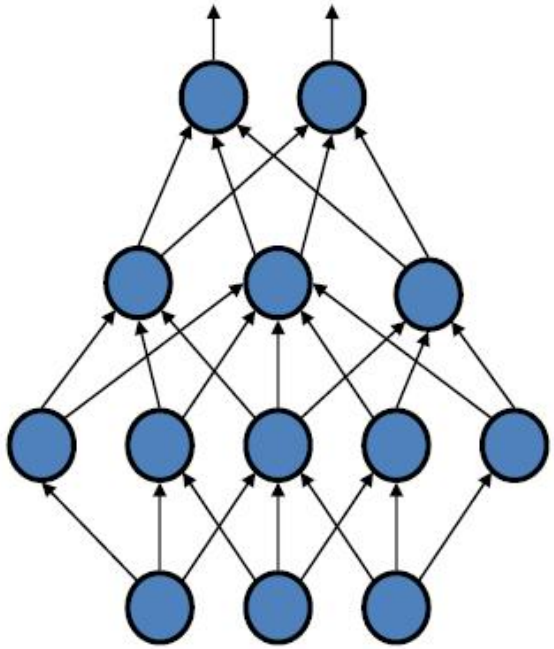
  Hinton, G. E., Salakutdinov, Reducting the dimensionality of data with neural networks. Science, 2006, 313: 504-507.

- **Autoencoders**

  Bengio, Y., Lamblin, P., Popovici, P., Larochelle, H. (2007). Greedy Layer-Wise Training of Deep Networks, Advances in Neural Information Processing Systems 19

2.3 Deep Neural Network

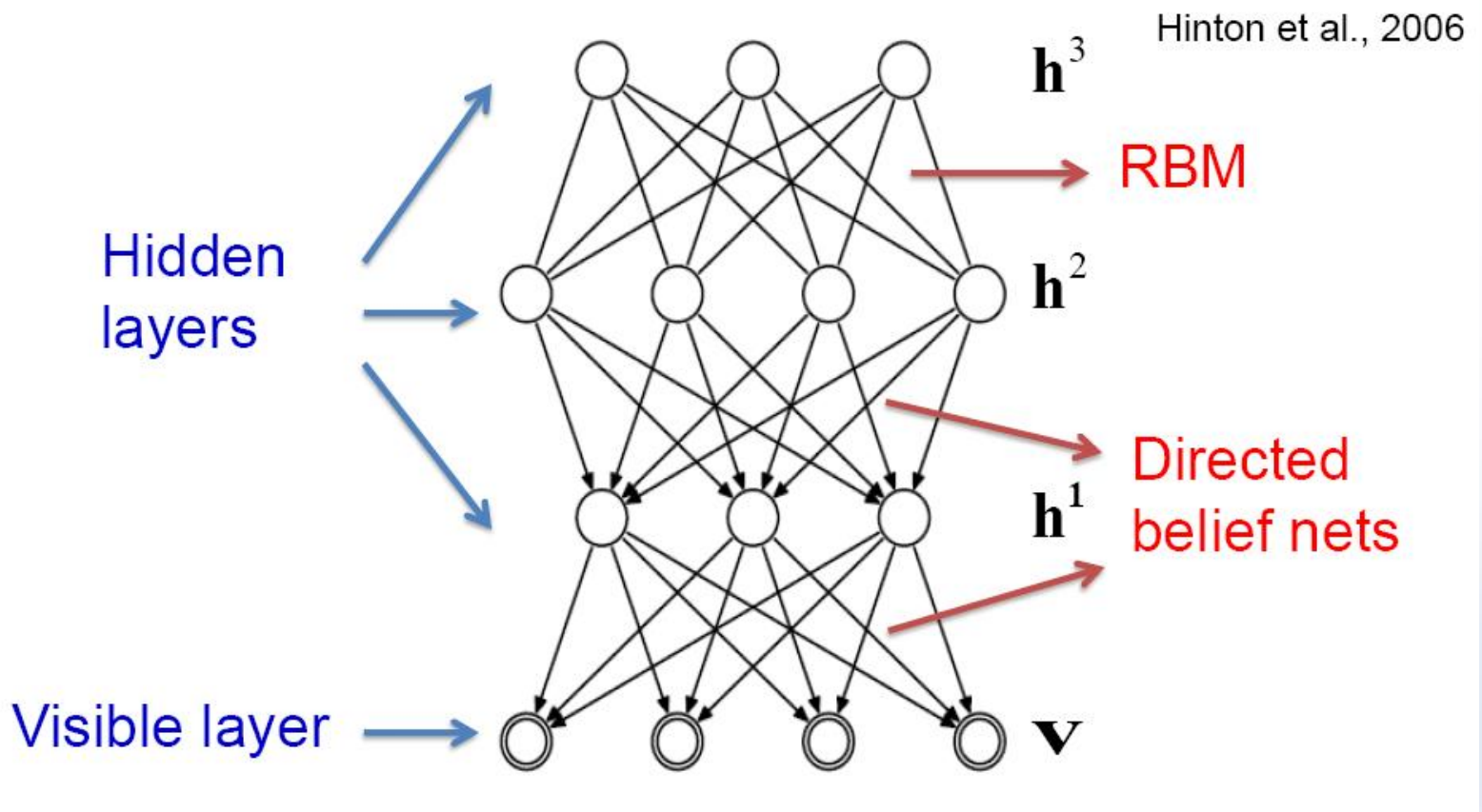
- Simple to construct
  - –Sigmoid nonlinearity for hidden layers
  - –Softmax for the output layer



Deep Belief Network (DBN)

- Probabilistic generative models
- Contain many layers of hidden variables
- Unsupervised pre-learning provides a good initialization of the network
  - –maximizing the lower-bound of the log-likelihood of the data （MLE in fact!）
- Supervised fine-tuning
  - –Generative: Up-down algorithm
  - –Discriminative: backpropagation

DBN Structure

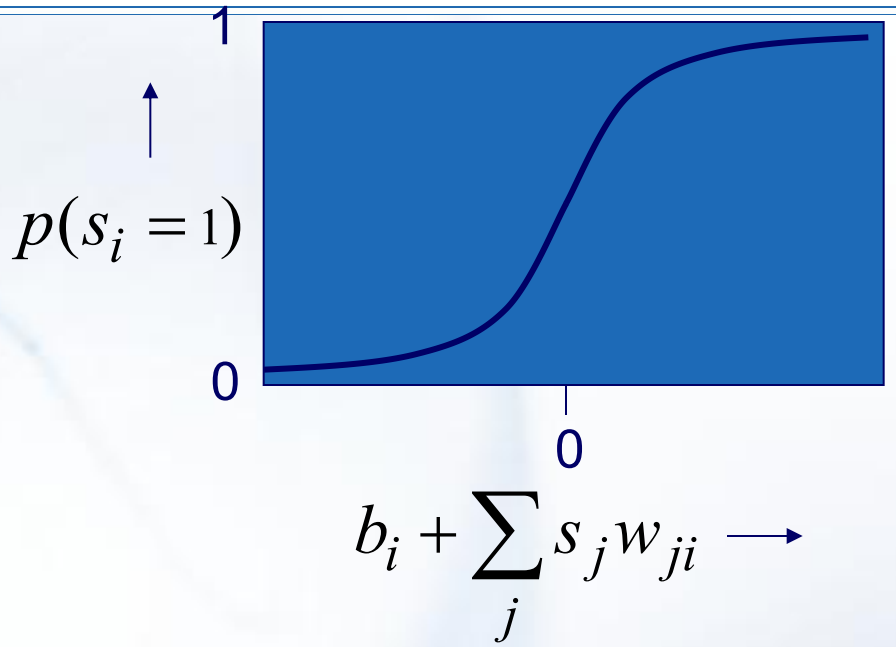


$$P(\mathbf{v},\mathbf{h}^1,\mathbf{h}^2,\ldots,\mathbf{h}^l) = P(\mathbf{v}\,|\,\mathbf{h}^1)P(\mathbf{h}^1\,|\,\mathbf{h}^2)\ldots P(\mathbf{h}^{l-2}\,|\,\mathbf{h}^{l-1})P(\mathbf{h}^{l-1},\mathbf{h}^l)$$

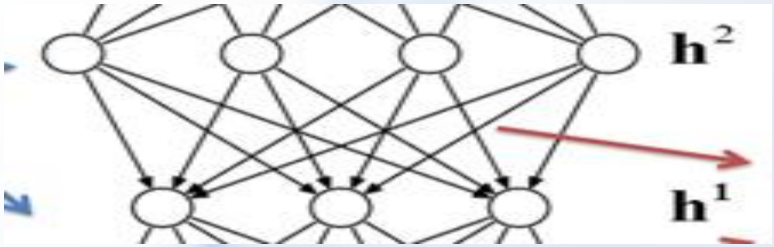
$$P\left(\mathbf{h}^i \middle| \mathbf{h}^{i+1}\right) = \prod_{j=1}^{n^i} P\left(h_j^i \middle| \mathbf{h}^{i+1}\right)$$

Stochastic binary units (Bernoulli variables)

- These have a state of 1 or 0.
- The probability of turning on is determined by the weighted input from other units (plus a bias)



$$P\left(h^i \middle| h^{i+1}\right) = \prod_{j=1}^{n^i} P\left(h_j^i \middle| h^{i+1}\right)$$



$$p(h_j^i = 1 | \mathbf{h}^{i+1}) = \frac{1}{1 + \exp(-b_j^i - \sum_{k=1}^{n^{i+1}} w_{kj}^i h_k^{i+1})}$$

Restricted Boltzmann Machines

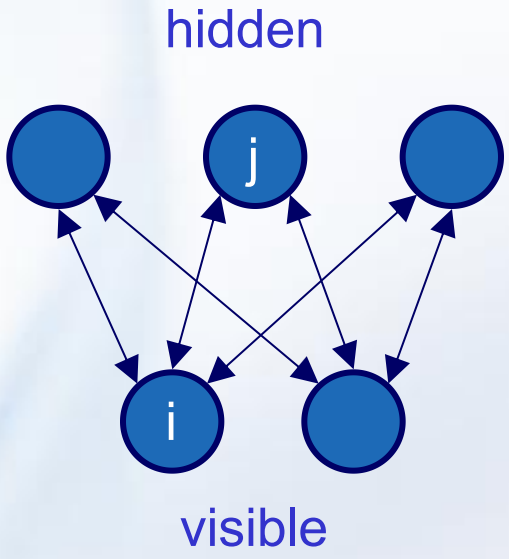
- An RBM has a single layer of hidden units which are not connected to each other and have undirected, symmetrical connections to a layer of visible units.

$$P(v,h) = \frac{1}{z} e^{\mathbf{h}'\mathbf{W}\mathbf{v}+\mathbf{b}'\mathbf{v}+\mathbf{c}'\mathbf{h}}$$

$$energy(v,h) = -\mathbf{h}'\mathbf{W}\mathbf{v} - \mathbf{b}'\mathbf{v} - \mathbf{c}'\mathbf{h}$$

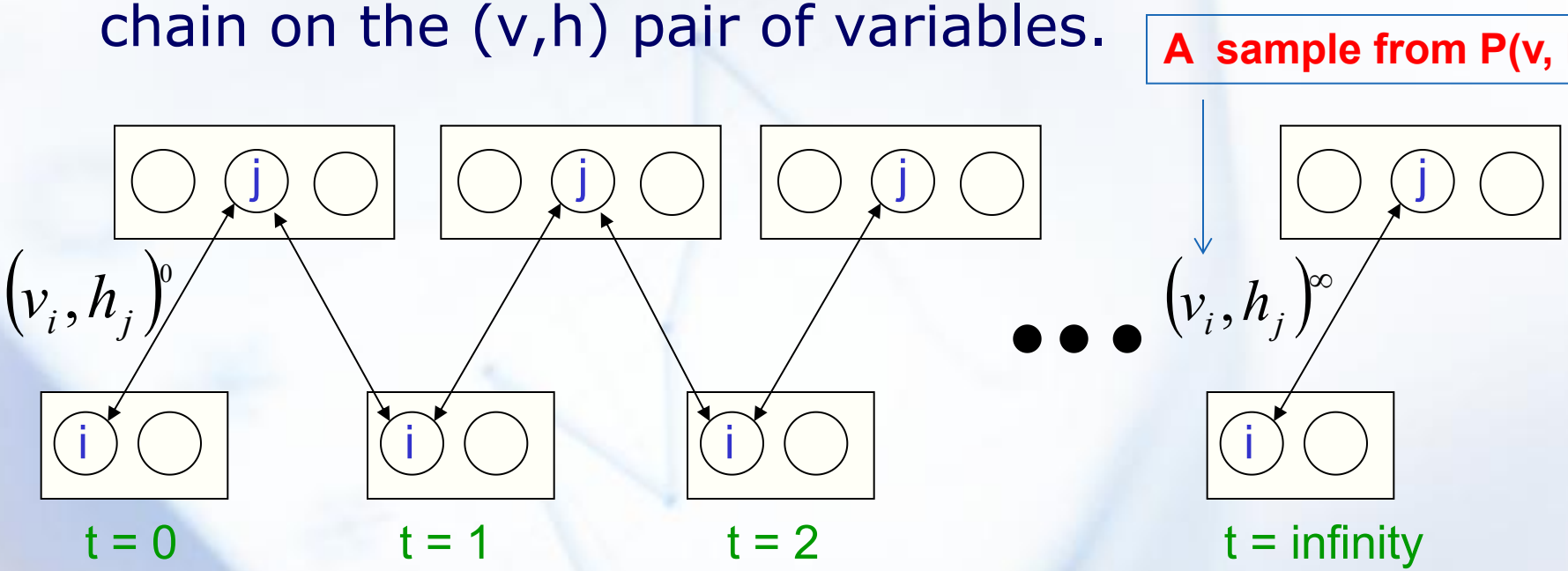
$$P(v_i = 1|\mathbf{h}) = sigm\left(b_i + \sum_k W_{ki} h_k\right)$$

$$Q(h_j = 1|\mathbf{v}) = sigm\left(c_j + \sum_k W_{jk} v_k\right)$$



MLE learning for RBM

- To obtain an estimator of the gradient on the log-likelihood of an RBM, we consider a Gibbs Markov chain on the (v,h) pair of variables.



Start with a training vector on the visible units.

Then alternate between updating all the hidden units in parallel and updating all the visible units in parallel.

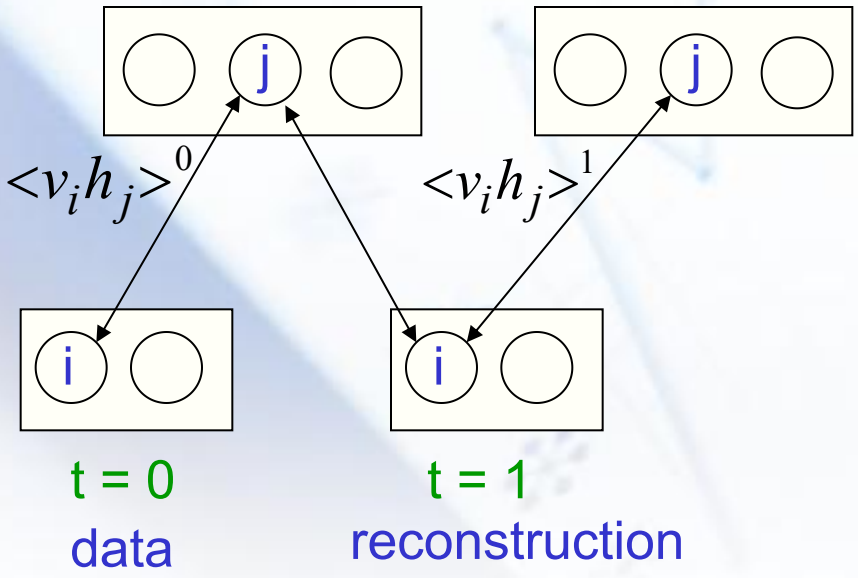
Gradient on the log-likelihood of an RBM

$$\log P(\mathbf{v}_0) = \log \sum_{\mathbf{h}} P(\mathbf{v}_0, \mathbf{h}) = \log \sum_{\mathbf{h}} e^{-energy(\mathbf{v}_0, \mathbf{h})} - \log \sum_{\mathbf{v},\mathbf{h}} e^{-energy(\mathbf{v}, \mathbf{h})}$$

$$\frac{\partial \log P(\mathbf{v}_0)}{\partial \boldsymbol{\theta}} = -\sum_{\mathbf{h}_0} Q(\mathbf{h}_0 | \mathbf{v}_0) \frac{\partial energy(\mathbf{v}_0, \mathbf{h}_0)}{\partial \boldsymbol{\theta}} + \sum_{\mathbf{v}_k, \mathbf{h}_k} P(\mathbf{v}_k, \mathbf{h}_k) \frac{\partial energy(\mathbf{v}_k, \mathbf{h}_k)}{\partial \boldsymbol{\theta}}$$

$$k \to \infty$$

- A quick way (Contrastive Divergence): to take k small (typically 1).



Start with a training vector on the visible units.

Update all the hidden units in parallel

Update the all the visible units in parallel to get a “reconstruction”.

Update the hidden units again.

RBM training algorithm

---

for all hidden units i **do**

- compute $Q(\mathbf{h}_{0i} = 1|\mathbf{v}_0)$ (for binomial units, $\text{sigm}(-b_i - \sum_j W_{ij}\mathbf{v}_{0j})$)
- sample $\mathbf{h}_{0i}$ from $Q(\mathbf{h}_{0i} = 1|\mathbf{v}_0)$

end for

for all visible units j **do**

- compute $P(\mathbf{v}_{1j} = 1|\mathbf{h}_0)$ (for binomial units, $\text{sigm}(-c_j - \sum_i W_{ij}\mathbf{h}_{0i})$)
- sample $\mathbf{v}_{1j}$ from $P(\mathbf{v}_{1j} = 1|\mathbf{h}_0)$

end for

for all hidden units i **do**

- compute $Q(\mathbf{h}_{1i} = 1|\mathbf{v}_1)$ (for binomial units, $\text{sigm}(-b_i - \sum_j W_{ij}\mathbf{v}_{1j})$)

end for

- $W \leftarrow W - \epsilon(\mathbf{h}_0\mathbf{v}_0' - Q(\mathbf{h}_{1.} = 1|\mathbf{v}_1)\mathbf{v}_1')$
- $b \leftarrow b - \epsilon(\mathbf{h}_0 - Q(\mathbf{h}_{1.} = 1|\mathbf{v}_1))$
- $c \leftarrow c - \epsilon(\mathbf{v}_0 - \mathbf{v}_1)$

Greedy Layer-wise Training of DBN

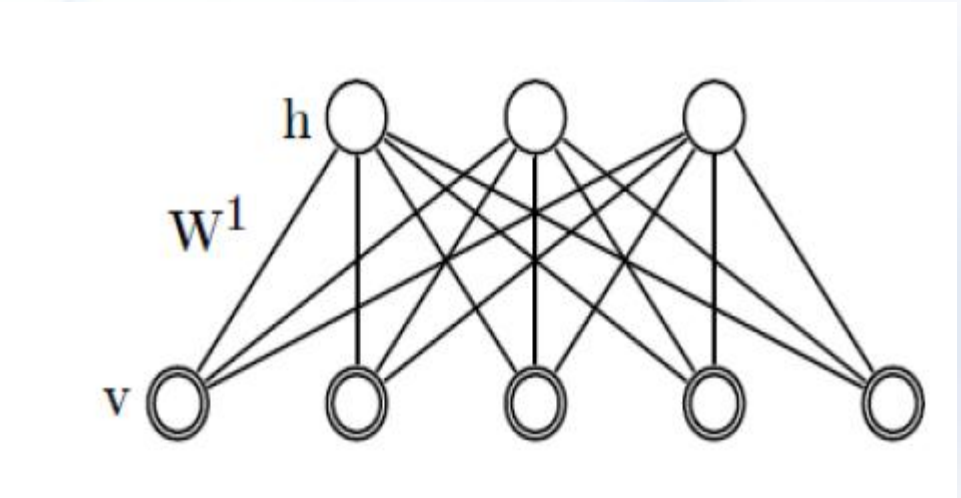
- **Basic Idea:**
  - 1-level DBN is an RBM, so use RBM training
  - After training the top-level RBM of a m-level DBN, insert the RBM parameters in a (m+1)-level DBN
  - $Q(\mathbf{h}^m|\mathbf{h}^{m-1})$ of the RBM does not correspond to $P(\mathbf{h}^m|\mathbf{h}^{m-1})$ in the DBN, except the top layer.
  - However, we can use $Q(\mathbf{h}^m|\mathbf{h}^{m-1})$ as the approximation of $P(\mathbf{h}^m|\mathbf{h}^{m-1})$
  - Variational bound justifies such greedy layer-wise training of RBMs: Improvement for the next layer will yield improvement for the previous layer

$$\log P(\mathbf{x}) \geq H_{Q(\mathbf{h}|\mathbf{x})} + \sum_{\mathbf{h}} Q(\mathbf{h}|\mathbf{x}) \left(\log P(\mathbf{h}) + \log P(\mathbf{x}|\mathbf{h})\right)$$

Trained by the second layer RBM

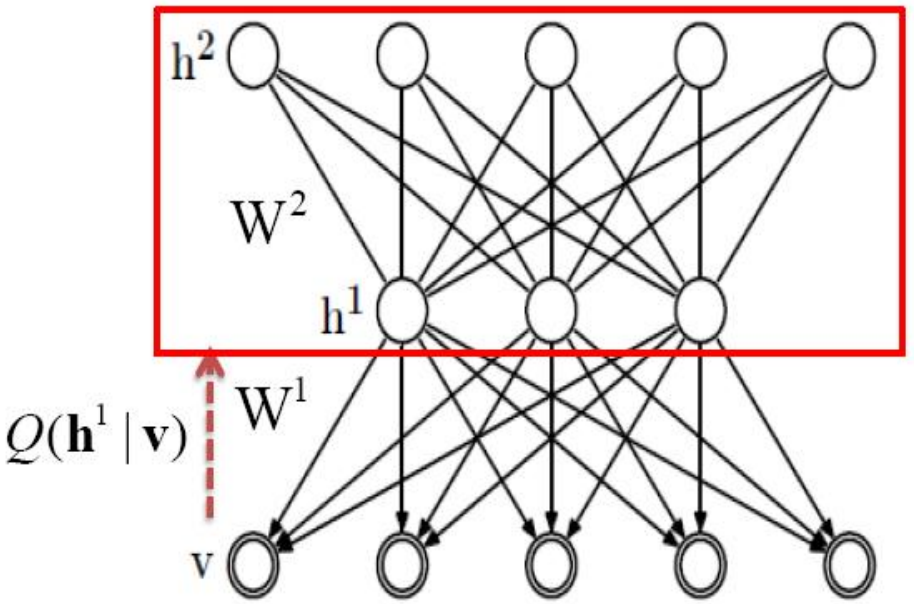
Greedy Layer-wise Training of DBN

- **First step:**
  - –Construct an RBM with an input layer v and a hidden layer h
  - –Train the RBM



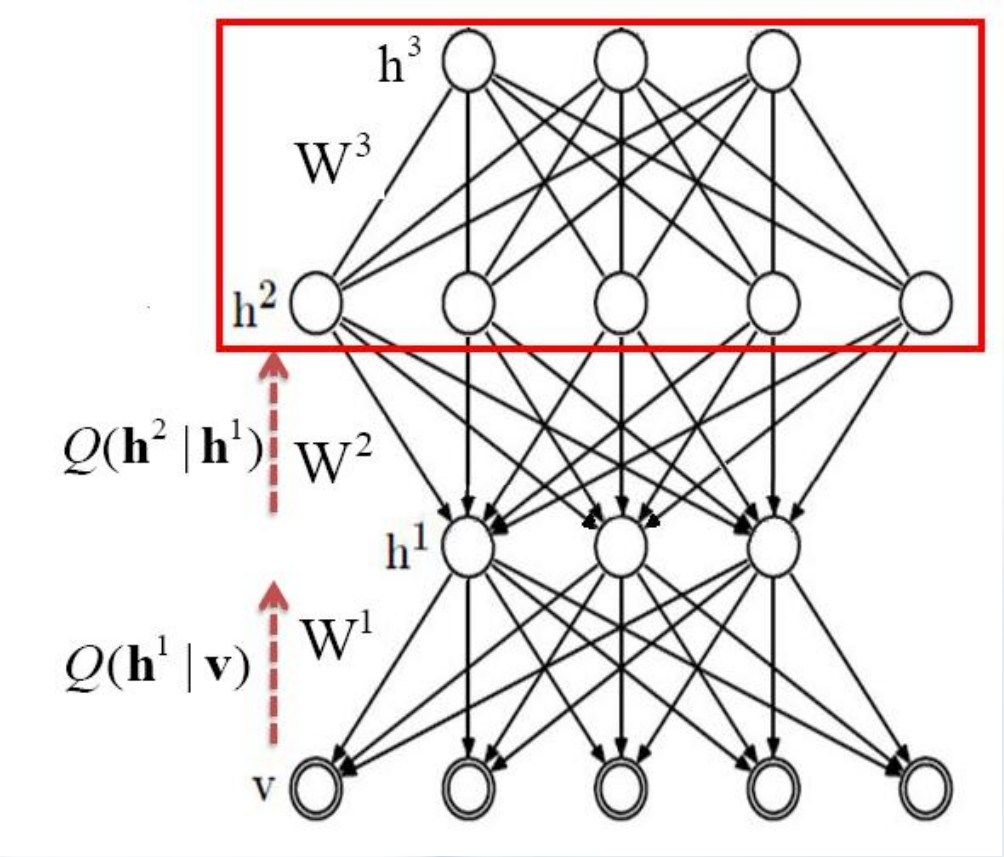
Greedy Layer-wise Training of DBN

- **Second step:**
  - –Stack another hidden layer on top of the RBM to form a new RBM
  - – Fix $\mathbf{W}^1$, sample $\mathbf{h}^1$ from $Q(\mathbf{h}^1 \mid \mathbf{v})$ as input. Train $\mathbf{W}^2$ as RBM.



Greedy Layer-wise Training of DBN

- **Third step:**
  - Continue to stack layers on top of the network, train it as previous step, with sample sampled from $Q(\mathbf{h}^2 \mid \mathbf{h}^1)$
- **Continued……**



Greedy Layer-wise Training of DBN

- initialize $b^0 = 0$

for $\ell = 1$ to L **do**

- initialize $W^i = 0$, $b^i = 0$

while not stopping criterion **do**

- sample $\mathrm{g}^0 = x$ from $\widehat{p}$

for $i = 1$ to $\ell - 1$ **do**

- sample g^i from $Q(\mathrm{g}^i | \mathrm{g}^{i-1})$

end for

- RBMupdate$(\mathrm{g}^{\ell-1}, \epsilon, W^\ell, b^\ell, b^{\ell-1})$

end while

end for

Fine-tuning

- Fine-tune the parameters of all the layers together.
- Treat DBN greedy layer-wise training as "pre-training" that finds a good initial set of weights which can be fine-tuned by a local search procedure.
  - Contrastive wake-sleep is one way of fine-tuning the model to be better at generation.
- Backpropagation can be used to fine-tune the model for better discrimination.
  - This overcomes many of the limitations of standard backpropagation.

Fine-tuning with up-down algorithm

Step1. Do a stochastic bottom-up pass

- Use the MLE rule to adjust the top-down weights to be good at reconstructing the feature activities in the layer below.

Step2. Do a few iterations of sampling in the top level RBM

-- Adjust the weights in the top-level RBM.

Step3. Do a stochastic top-down pass

- Adjust the bottom-up weights to be good at reconstructing the feature activities in the layer above.

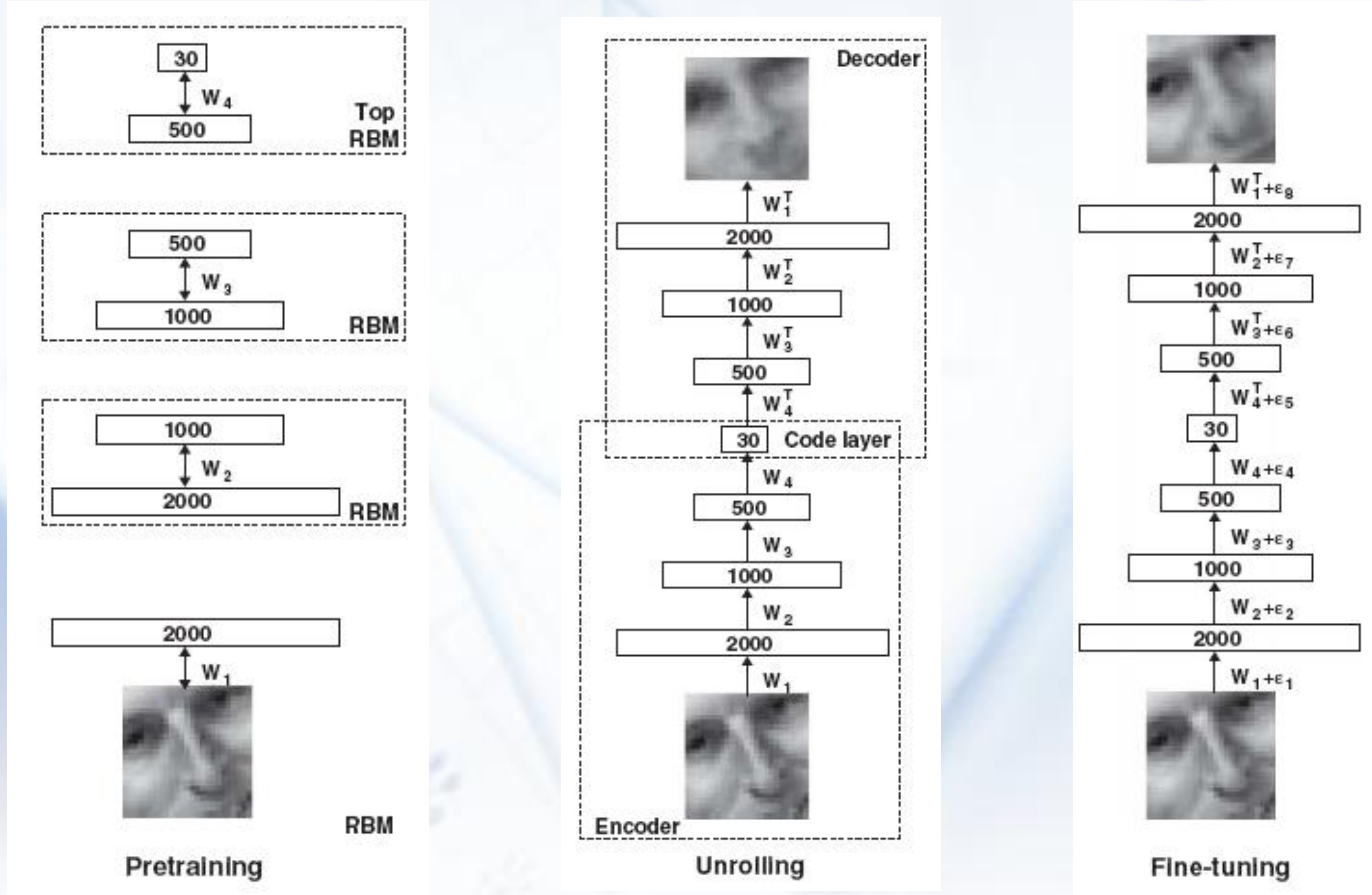
Fine-tuning with back-propagation

- Optimize all the parameters with respect to the supervised criterion C, using stochastic gradient descent.

- Recursively define mean-field propagation $\mu^i(x) = E[\mathbf{g}^i | \mathbf{g}^{i-1} = \mu^{i-1}(x)]$ where $\mu^0(x) = x$, and $E[\mathbf{g}^i | \mathbf{g}^{i-1} = \mu^{i-1}]$ is the expected value of $\mathbf{g}^i$ under the RBM conditional distribution $Q(\mathbf{g}^i | \mathbf{g}^{i-1})$, when the values of $\mathbf{g}^{i-1}$ are replaced by the mean-field values $\mu^{i-1}(x)$. In the case where $\mathbf{g}^i$ has binomial units, $E[\mathbf{g}^i_j | \mathbf{g}^{i-1} = \mu^{i-1}] = \mathrm{sigm}(-b^i_j - \sum_k W^i_{jk} \mu^{i-1}_k(x))$.
- Define the network output function $f(x) = V(\mu^L(x)', 1)'$
- Iteratively minimize the expected value of $C(f(x), y)$ for pairs (x, y) sampled from $\widehat{p}$ by tuning parameters W, b, V. This can be done by stochastic gradient descent with learning rate ϵ_C, using an appropriate stopping criterion such as early stopping on a validation set.

AutoEncoder

- The weights can be trained together by minimizing the discrepancy between the original data and its reconstruction.



Greedy AutoEncoder Training Algorithm

Goal of AutoEncoder Training: accurately reconstruct each training data

Goal of RBM Training: make the distribution of the confabulations be the same as the distribution of the data

- initialize $b^0 = 0$.
- define $\mu^0(x) = x$.

for $\ell = 1$ to L **do**

- initialize $b^\ell = 0$.
- initialize temporary parameter vector $c^\ell = 0$.
- initialize W^ℓ by sampling from uniform$(-a, a)$, with $a = 1/n^{\ell-1}$.
- define the ℓ-th hidden layer output $\mu^\ell(x) = \text{sigm}(b^\ell + W^\ell \mu^{\ell-1}(x))$.
- define the ℓ-th hidden layer reconstruction parameter function, e.g. in the binomial case $\theta^\ell = \text{sigm}(c^\ell + W^{\ell'} \mu^\ell(x))$ is the vector of probabilities for the each bit to take value 1.

while not stopping criterion **do**

for $i = 1$ to $\ell - 1$ **do**

- compute $\mu^i(x)$ from $\mu^{i-1}(x)$.

end for

- compute $\mu^\ell(x)$ from $\mu^{\ell-1}(x)$.
- compute reconstruction probability parameters θ^ℓ from $\mu^\ell(x)$.
- compute the error C in reconstructing $\mu^{\ell-1}$ from probability with parameters θ^ℓ.
- compute $\frac{\partial C}{\partial \omega}$, for $\omega = (W^\ell, b^\ell, c^\ell)$
- update layer parameters: $\omega \leftarrow \omega - \epsilon \frac{\partial C}{\partial \omega}$

end while

end for

Example: Digit Recognition on MNIST

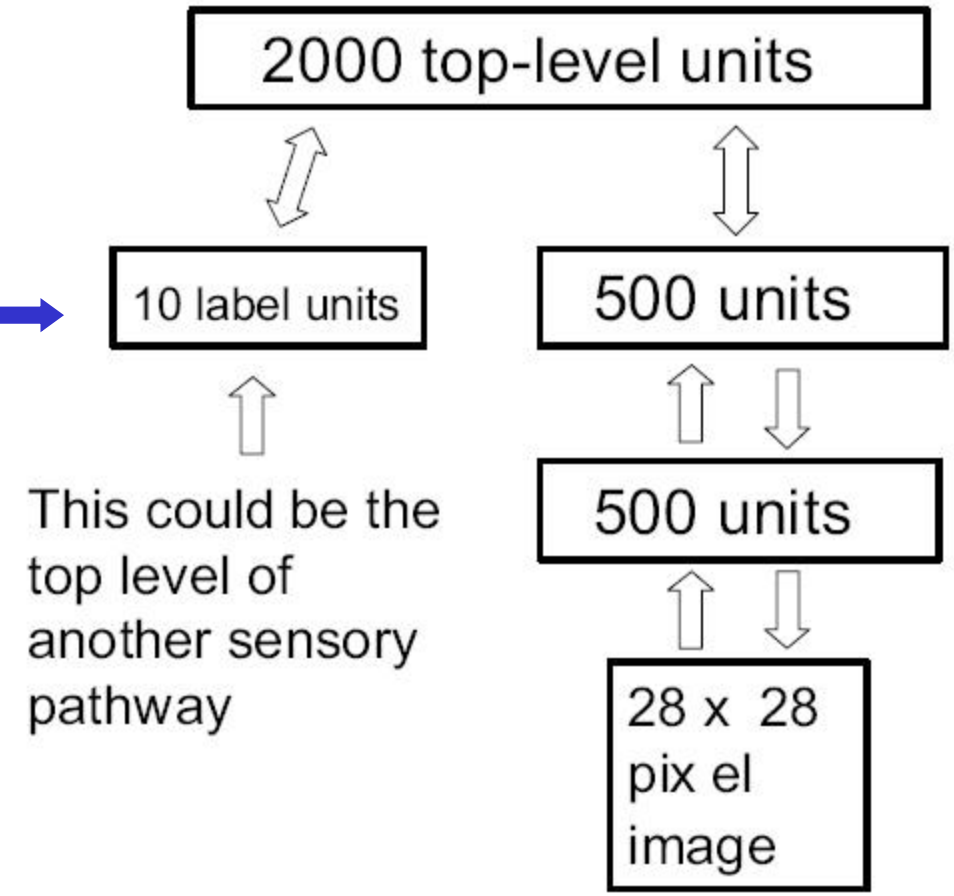
On the original data

- Very carefully trained backprop net with one or two hidden layers (Platt; Hinton) — 1.6%
- SVM (Decoste & Schoelkopf, 2002) — 1.4%
- Generative model of joint density of images and labels (+ generative fine-tuning) — 1.25%
- Generative model of unlabelled digits followed by gentle backpropagation (Hinton & Salakhutdinov, Science 2006) — 1.2%

DBN for digit recognition

The top two layers form an associative memory whose energy landscape models the low dimensional manifolds of the digits.

The energy valleys have names



The model learns to generate combinations of labels and images.

To perform recognition we start with a neutral state of the label units and do an up-pass from the image followed by a few iterations of the top-level associative memory.

Stage1: Greedy Layer-wise Training

Stage2: fine-tuning

Looking into the 'mind' of the machine

Samples generated by letting the associative memory run with one label clamped.



Looking into the 'mind' of the machine

Test images and corresponding reconstruction

Test images →

By Autoencoder →

By PCA →

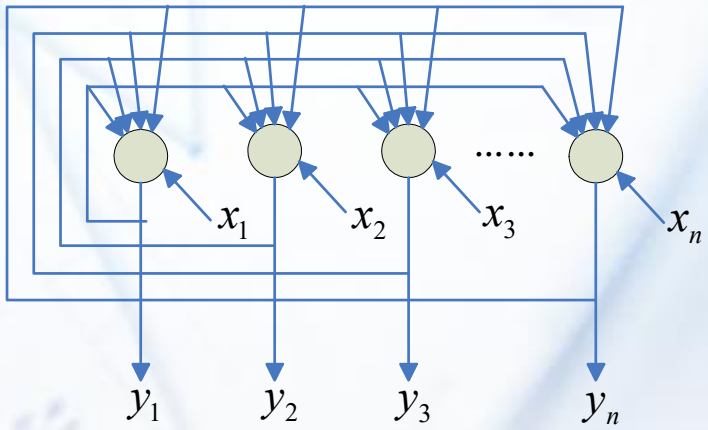


Part Ⅲ: Recurrent Network

3.1 Hopfield network

- **Feedback (recurrent) architecture**
- **Dynamic System**
- **Two types**

  Discrete (1982) and Continuous (*science*, 1984), by Hopfield



Attractors and Stability

- How do we "program" the solutions of the problem into stable states (attractors) of the network?
- How do we ensure that the feedback system designed is stable?

Lyapunov's modern stability theory allows us to investigate the stability problem by making use of a continuous scalar function of the state vector, called a Lyapunov (Energy) Function.

Energy Function for Hopfield Network

- **With input**

$$E = -\frac{1}{2}\sum_{i=0}^{n}\sum_{j=0}^{n}\omega_{ij}s_i s_j - \sum_{i=1}^{n} I_i s_i$$

- **Without input**

$$E = -\frac{1}{2}\sum_{i=0}^{n}\sum_{j=0}^{n}\omega_{ij}s_i s_j = -\frac{1}{2}\boldsymbol{S}^T\boldsymbol{\omega}\boldsymbol{S}$$

Hopfield model

- It is proven by Hopfield that asynchronous Hopfield network with

$$w_{ij} = \begin{cases} w_{ji}, i \neq j \\ 0, i = j \end{cases}$$

is stable. How?

Whatever be the initial state of the network, the energy decreases continuously with time until the system settles down into any local minimum of the energy surface.

Hopfield Network Application: *Associative Memories*

- A primary application of Hopfield network
- Recalls data (attractor state) based on an input that partially resembles the data itself.
- Also named content-addressable memory.



Stored Pattern



Memory Association

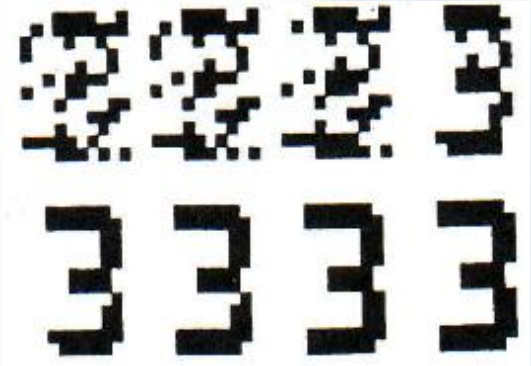
虞台文, *Feedback Networks and Associative Memories*

Hopfield Network: *Associative Memories*

- A primary application of Hopfield network
- Recalls data (attractor state) based on an input that partially resembles the data itself.
- Also named content-addressable memory.



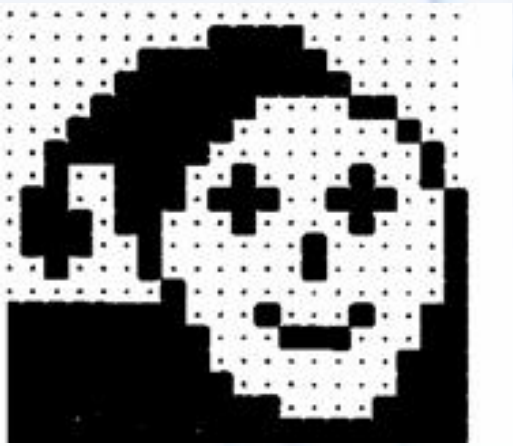
Stored Pattern



Memory Association

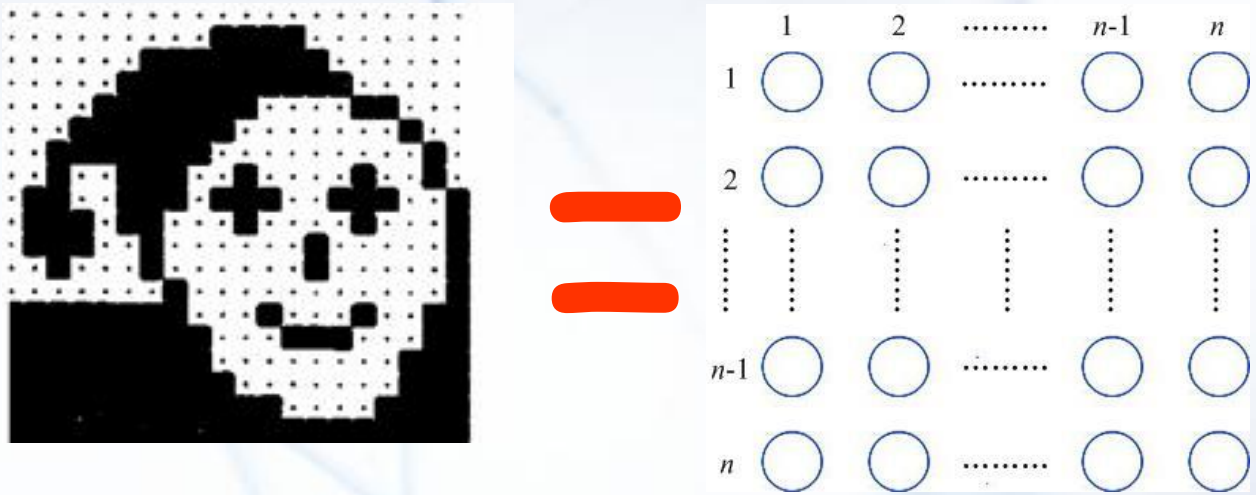
From 虞台文, *Feedback Networks and Associative Memories*

How to store patterns?



=?

How to store patterns?



n^2 : Dimension of the stored pattern

Weight determination: Outer Product

- **Vector form:**

$$\omega = \sum_{k=1}^{K} X_k X_k^T - KI$$

- **Element form:**

$$\omega_{ij} = \begin{cases} \sum_{k=1}^{K} x_{ik} x_{jk}, i \neq j \\ 0, i = j \end{cases}$$

- **Why?**

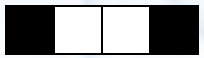
  Satisfy the Hopfield model

An example of Hopfield memory

From 虞台文, *Feedback Networks and Associative Memories*

$$\omega = \sum_{k=1}^{K} X_k X_k^T - KI$$

$$\mathbf{x}^1 = (1,-1,-1,1)^T$$



$$\mathbf{x}^2 = (-1,1,-1,1)^T$$



$$\mathbf{W} = \begin{bmatrix} 0 & -2 & 0 & 0 \\ -2 & 0 & 0 & 0 \\ 0 & 0 & 0 & -2 \\ 0 & 0 & -2 & 0 \end{bmatrix}$$

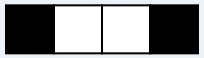
$$\mathbf{x}^1(\mathbf{x}^1)^T = \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{x}^2(\mathbf{x}^2)^T = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{x}^1(\mathbf{x}^1)^T + \mathbf{x}^2(\mathbf{x}^2)^T = \begin{bmatrix} 2 & -2 & 0 & 0 \\ -2 & 2 & 0 & 0 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & -2 & 2 \end{bmatrix}$$

The example contd.

$$\mathbf{x}^1 = (1,-1,-1,1)^T$$



$$\mathbf{x}^2 = (-1,1,-1,1)^T$$





$$\mathbf{W} = \begin{bmatrix} 0 & -2 & 0 & 0 \\ -2 & 0 & 0 & 0 \\ 0 & 0 & 0 & -2 \\ 0 & 0 & -2 & 0 \end{bmatrix}$$

$$E(\mathbf{x}) = -\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} w_{ij}x_i x_j - \sum_{i=1}^{n} I_i x_i$$

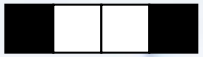
$$= -\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} w_{ij}x_i x_j$$

$$= -\frac{1}{2}\mathbf{x}^T\mathbf{W}\mathbf{x}$$

$$= 2(x_1x_2 + x_3x_4)$$

Recall the first pattern (x^1)

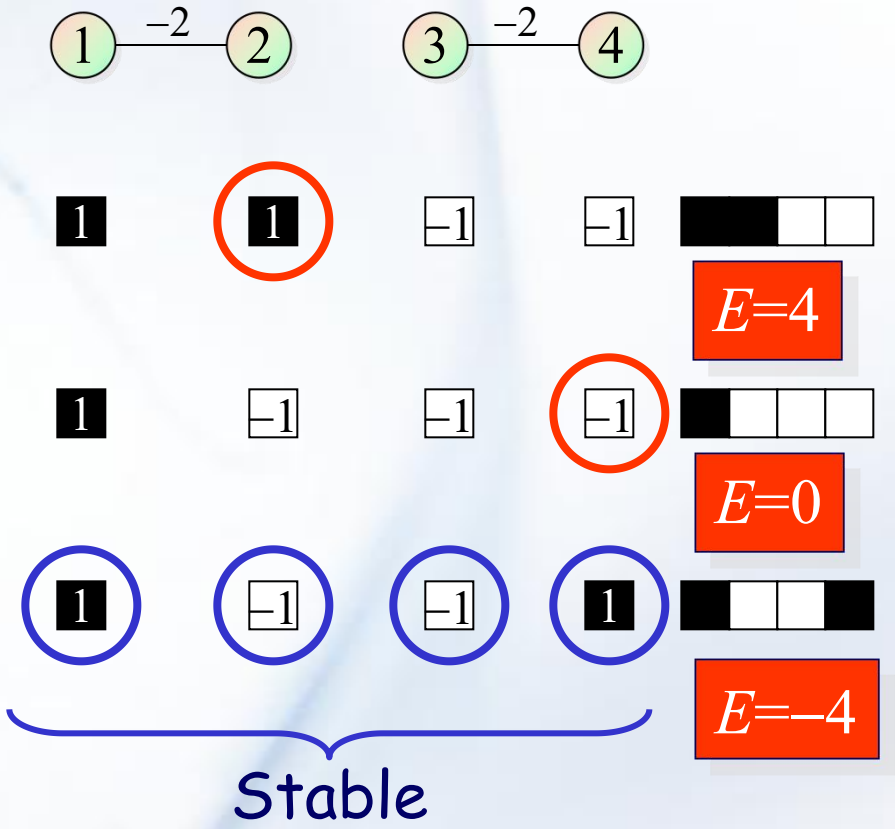
$$\mathbf{x}^1 = (1,-1,-1,1)^T$$



$$\mathbf{x}^2 = (-1,1,-1,1)^T$$



$$E(\mathbf{x}) = 2(x_1x_2 + x_3x_4)$$

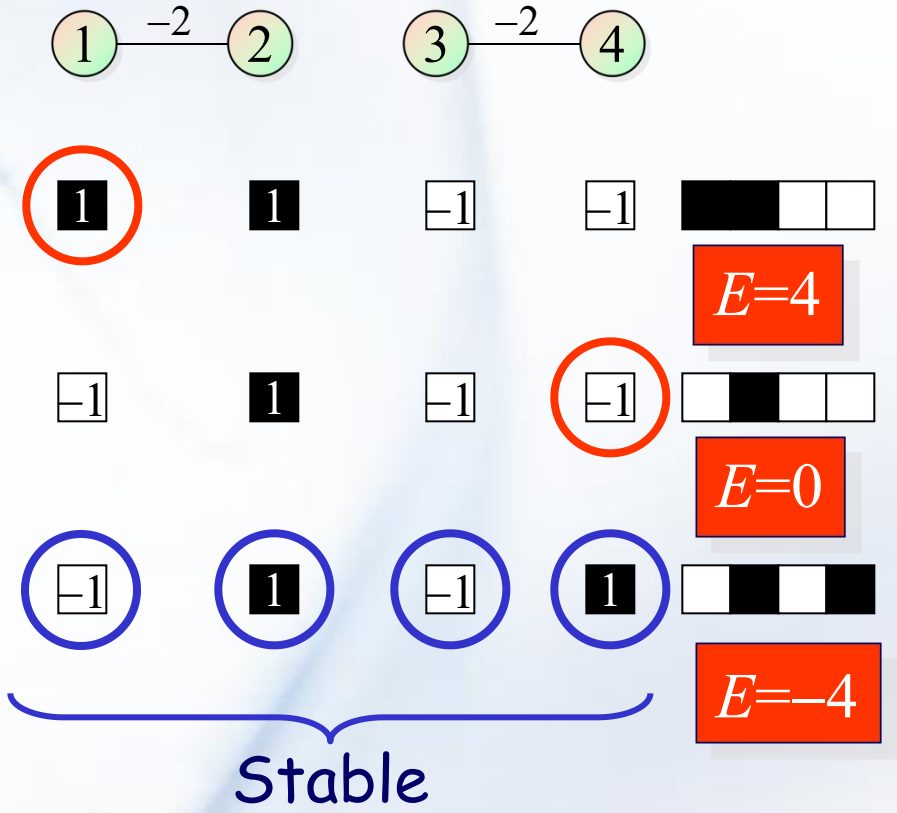


Recall the second pattern (x^2)

$$\mathbf{x}^1 = (1,-1,-1,1)^T$$

$$\mathbf{x}^2 = (-1,1,-1,1)^T$$

$$E(\mathbf{x}) = 2(x_1x_2 + x_3x_4)$$



Hopfield Network Application: *Combinatorial Optimization*

- Another primary application of Hopfield network
- Transform the objective function into the energy function. Then the attractor state of the network is the solution of the optimization problem.

An example: Solve Traveling Salesman Problem (TSP)

Given n cities with distances d_{ij}, what is the shortest tour?

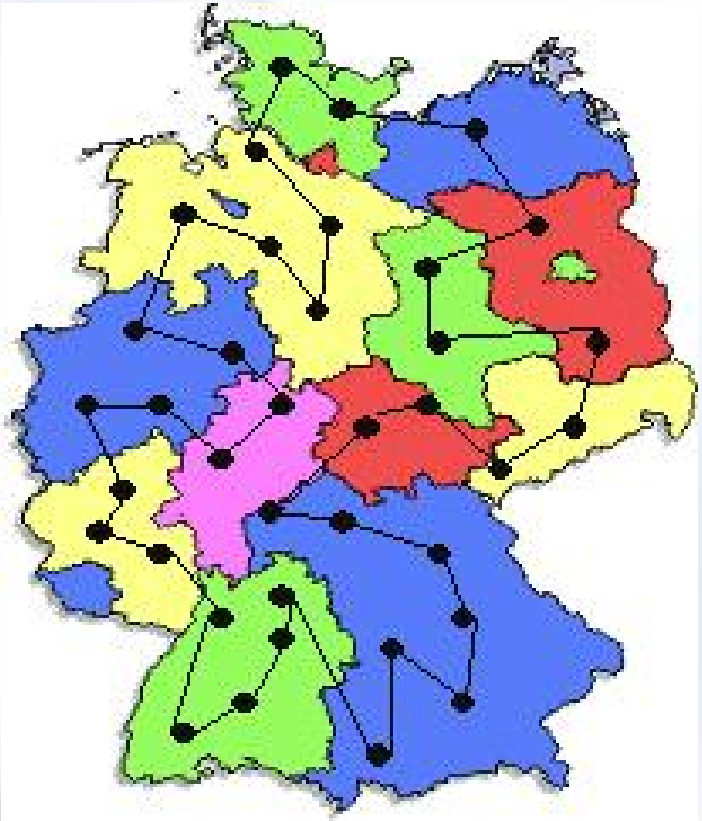
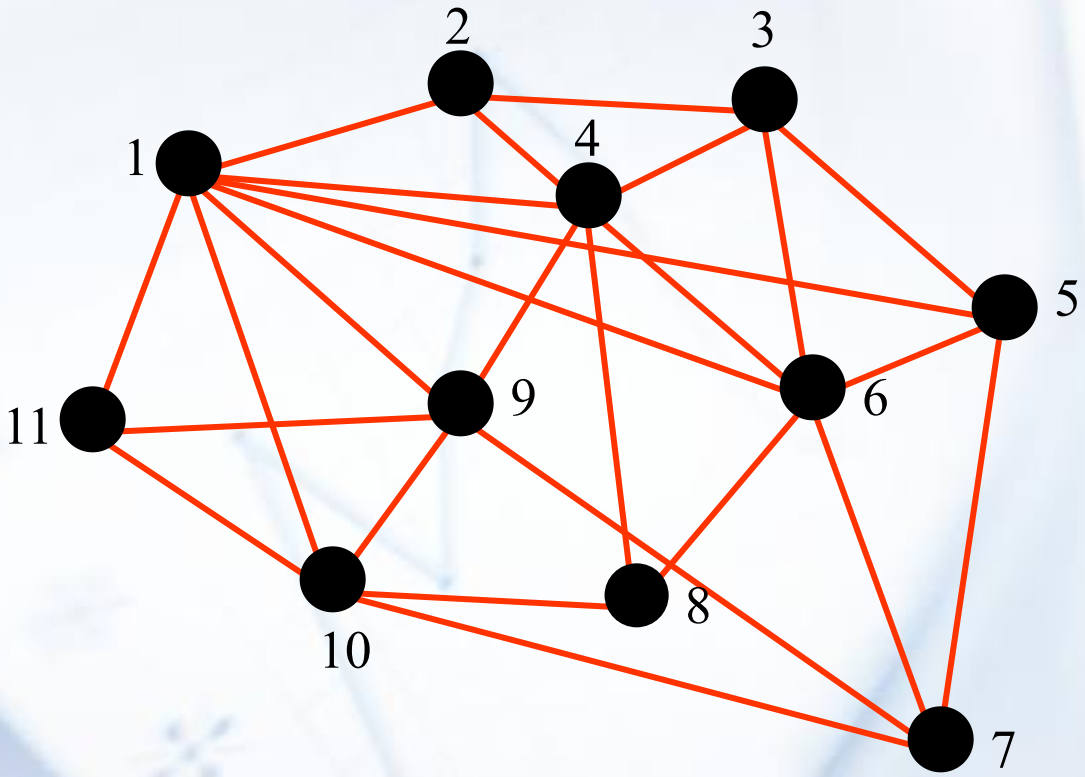
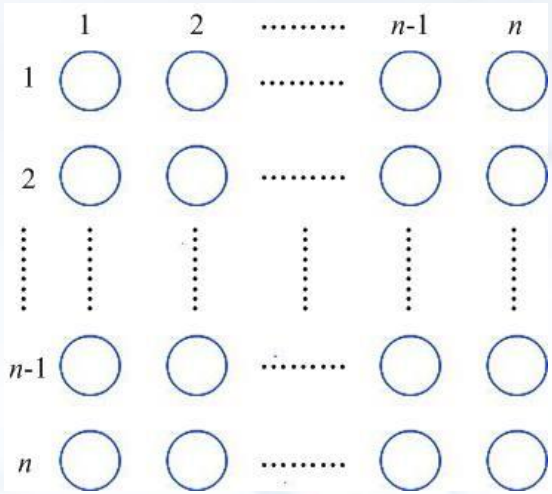


Illustration of TSP Graph

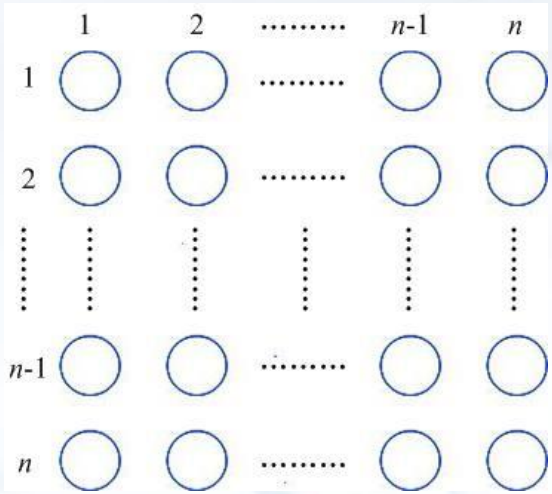


Hopfield Network for TSP



=?

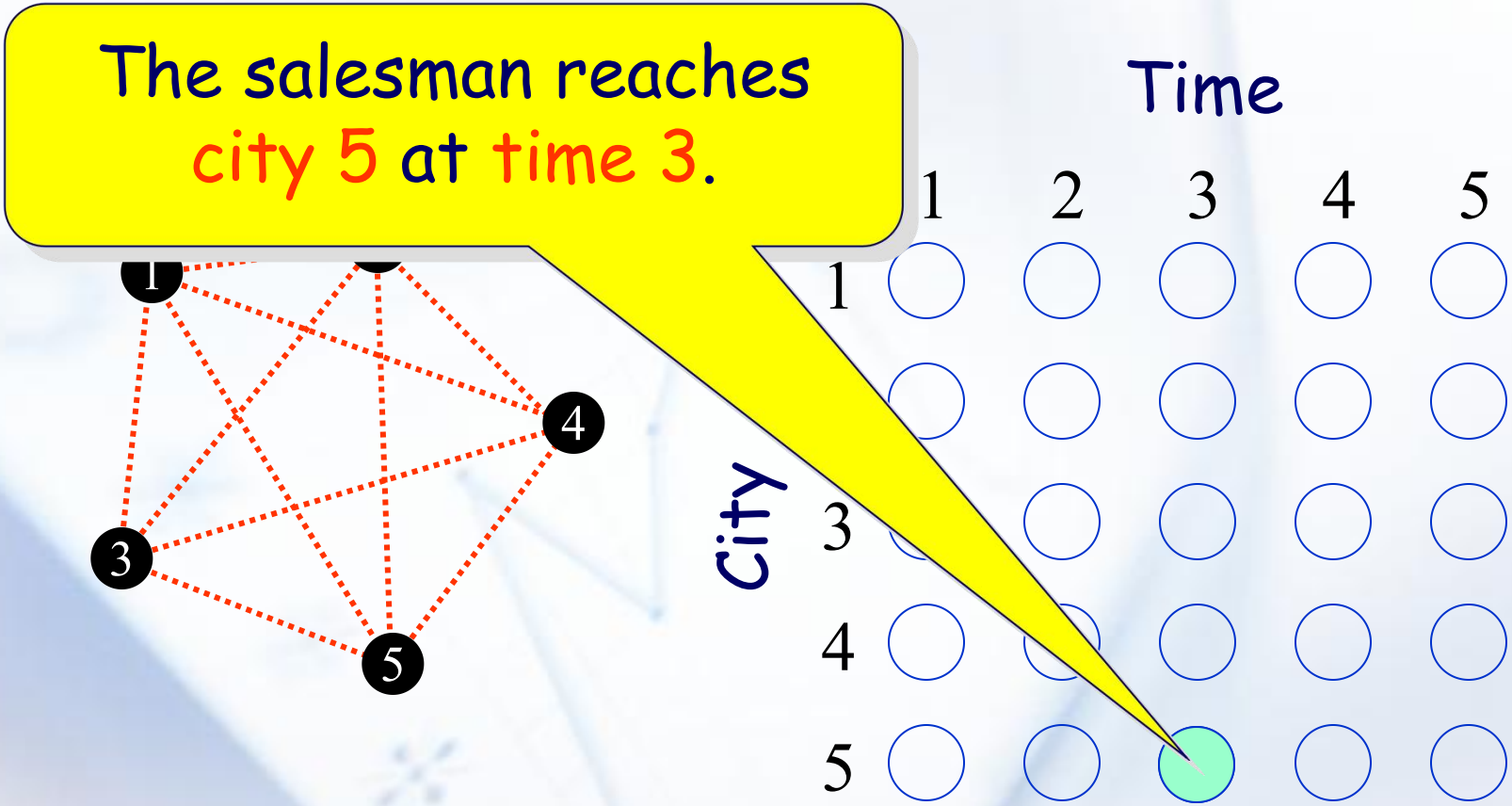
Hopfield Network for TSP



= City matrix

- Constraint
  1. Each row can have only one neuron “on”.
  2. Each column can have only one neuron “on”.
  3. For a *n*-city problem, *n* neurons will be on.

Hopfield Network for TSP



Weight determination for TSP: *Design Energy Function*

$$E = \underbrace{\frac{\lambda_1}{2}\sum_{i=1}^{n}\sum_{x=1}^{n}\sum_{y=1}^{n} s_{xi} d_{xy} \left(s_{y,i-1} + s_{y,i+1}\right)}_{goal}$$

$$+ \underbrace{\frac{\lambda_2}{2}\sum_{x=1}^{n}\sum_{i=1}^{n}\sum_{j=1}^{n} s_{xi} s_{xj} + \frac{\lambda_3}{2}\sum_{i=1}^{n}\sum_{x=1}^{n}\sum_{y=1}^{n} s_{xi} s_{yi} + \frac{\lambda_4}{2}\left(\sum_{x=1}^{n}\sum_{i=1}^{n} s_{xi} - n\right)^2}_{constraint}$$

Constraint-1

Each row can have only one neuron “on”.

Constraint-2

Each column can have only one neuron “on”.

Constraint-3

For a *n*-city problem, *n* neurons will be on.

Transform Energy Function into the form of 2D Hopfield Network

$$E = -\frac{1}{2}\sum_{i=0}^{n}\sum_{j=0}^{n}\omega_{ij}s_i s_j - \sum_{i=1}^{n} I_i s_i$$

$$E = -\frac{1}{2}\sum_{x=1}^{n}\sum_{i=1}^{n}\sum_{y=1}^{n}\sum_{j=1}^{n} s_{xi}\omega_{xi,yj}s_{yj} - \frac{1}{2}\sum_{x=1}^{n}\sum_{i=1}^{n} I_{xi}s_{xi}$$

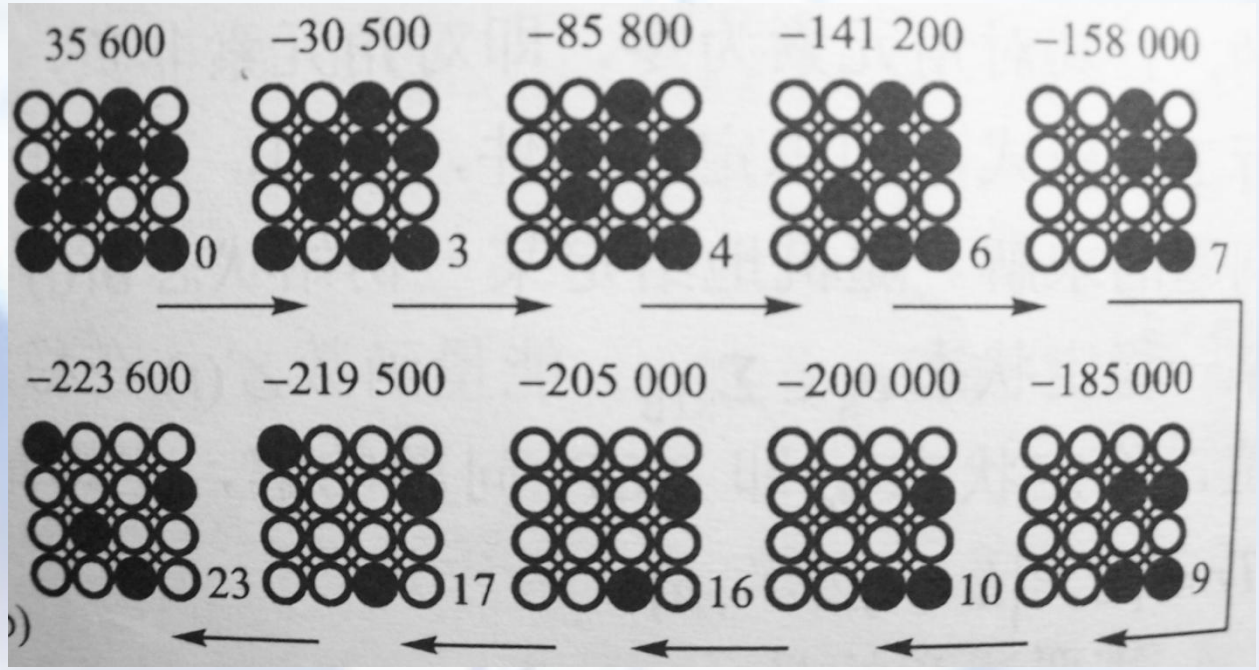
$$\omega_{xi,yj} = -\lambda_1 d_{xy}\left(\delta_{i,j+1} + \delta_{i,j-1}\right) - \lambda_2\delta_{xy}\left(1-\delta_{ij}\right) - \lambda_3\delta_{ij}\left(1-\delta_{xy}\right) - \lambda_4\left(1-\delta_{ij}\right)\left(1-\delta_{xy}\right)$$

$$I_{xi} = \lambda_4 n \qquad \delta_{xy} = \begin{cases} 1, x = y \\ 0, x \neq y \end{cases}$$

Network is built!

Iteration of Hopfield network for TSP problem

- The *initial state* generated randomly goes to the *stable state* (solution) with *minimum energy*

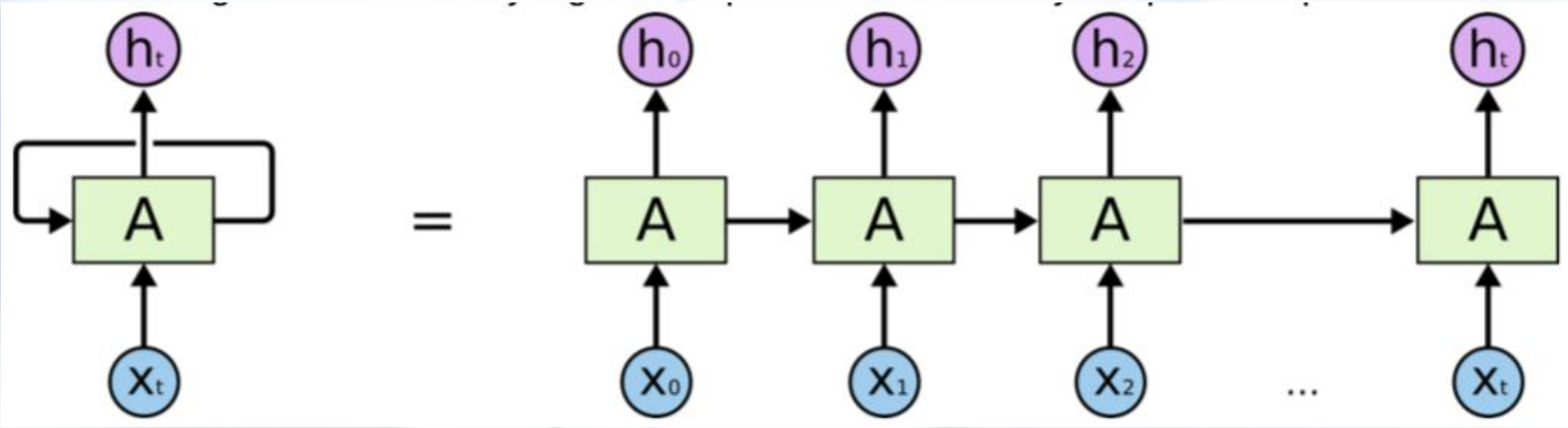


A 4-city example

From 阮晓刚，*神经计算科学*，**2006**

3.2 Long Short-Term Memory

We can unroll a RNN in time, in this way it can be considered as a deep neural network with indefinitely many layers and can be trained by performing Back-Propagation Through Time (BPTT).

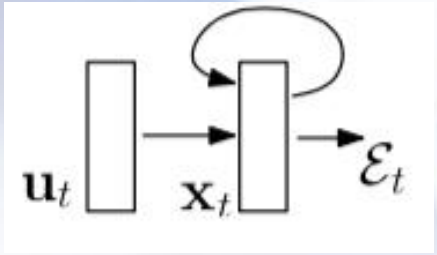


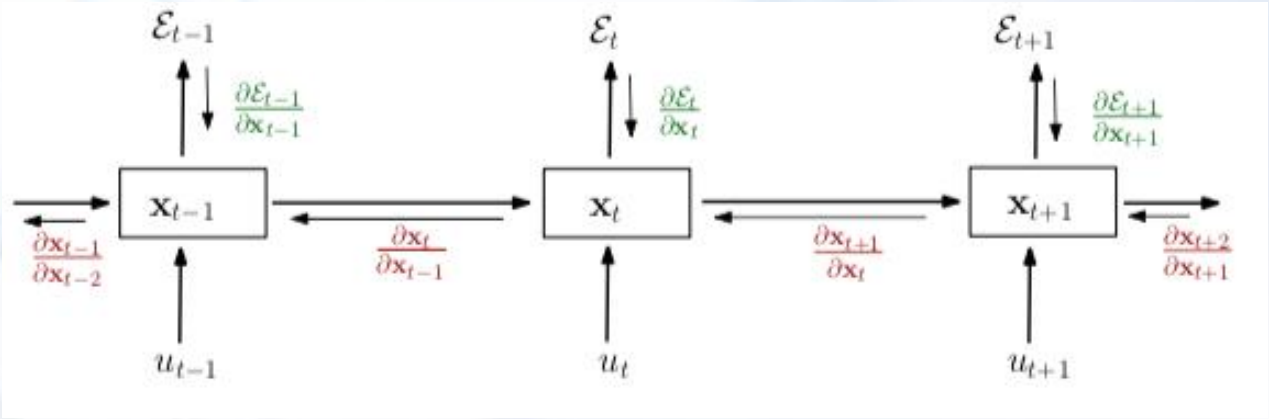
Back-Propagation Through Time (BPTT)

$$\frac{\partial \varepsilon}{\partial \theta} = \sum_{1 \le t \le T} \frac{\partial \varepsilon_t}{\partial \theta}$$

$$\frac{\partial \varepsilon_t}{\partial \theta} = \sum_{1 \le k \le t} \frac{\partial \varepsilon_t}{\partial x_t} \frac{\partial x_t}{\partial x_k} \frac{\partial x_k}{\partial \theta}$$

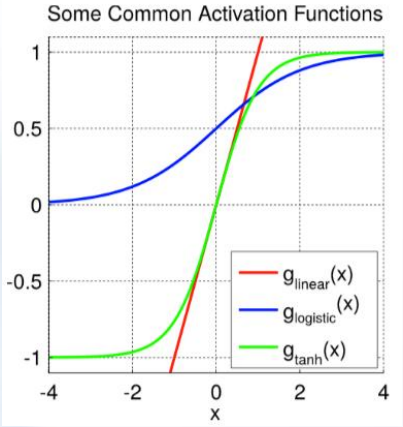
$$\frac{\partial x_t}{\partial x_k} = \prod_{t \ge i > k} \frac{\partial x_i}{\partial x_{i-1}} = \prod_{t \ge i > k} w_{rec}^T \, diag\left(\sigma'\left(x_{i-1}\right)\right)$$

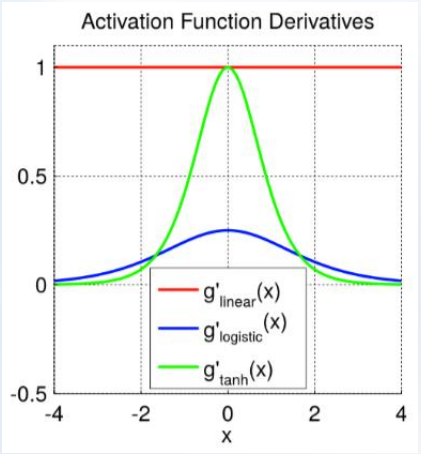




Vanishing/Exploding Gradients Problem

$$\frac{\partial \varepsilon_t}{\partial \theta} = \sum_{1 \le k \le t} \frac{\partial \varepsilon_t}{\partial x_t} \left(\prod_{t \ge i > k} w_{rec}^T \, diag\left(\sigma'\left(x_{i-1}\right)\right) \right) \frac{\partial x_k}{\partial \theta}$$

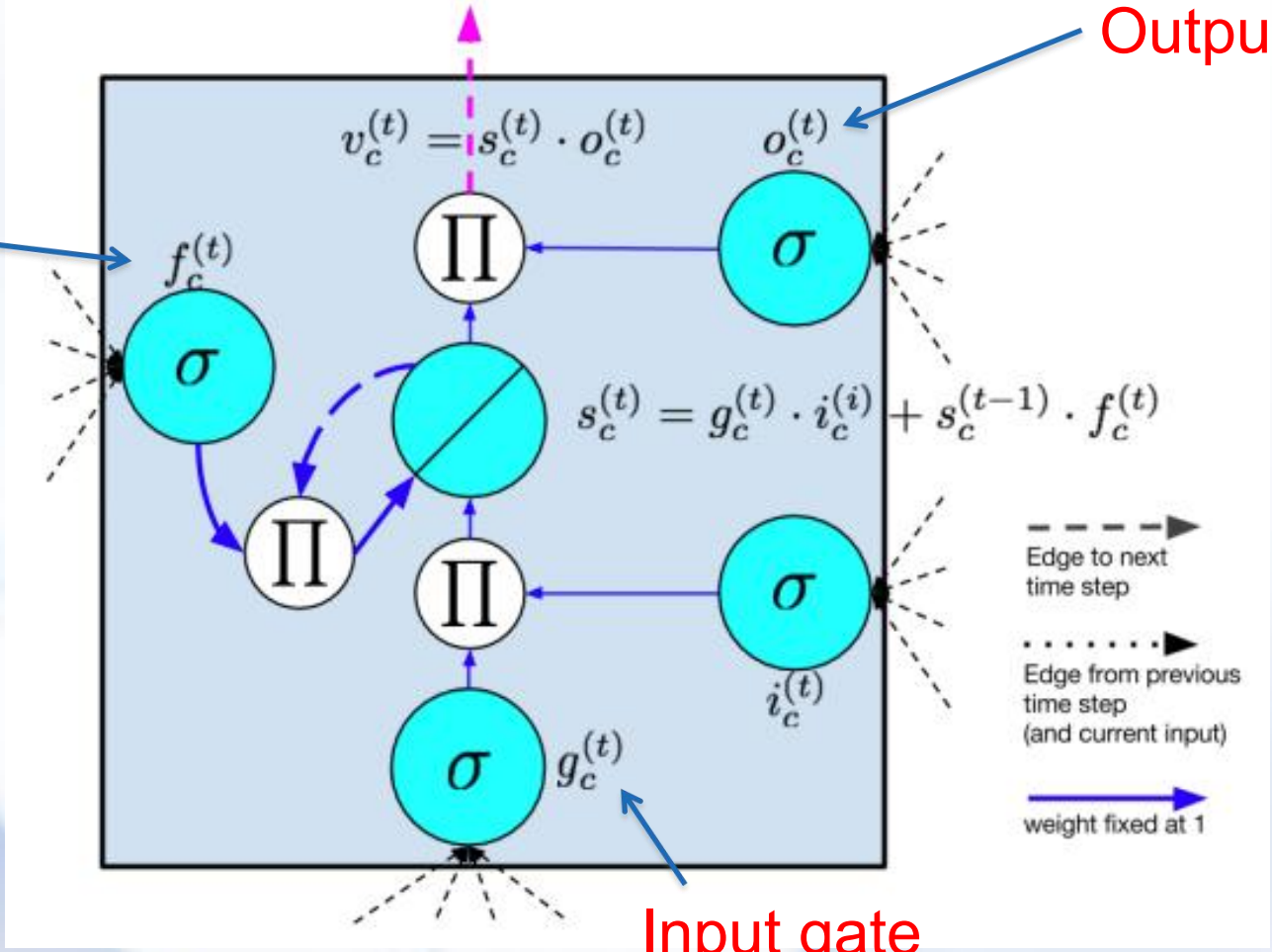




- tanh activation funciton maps all values into a range between -1 and 1, and the derivative is bounded by 1.
- With multiple matrix multiplications, gradient values shrink exponentially, gradient contributions from “far away” steps become zero.
- Depending on activation functions and network parameters, gradients could explode instead of vanishing.

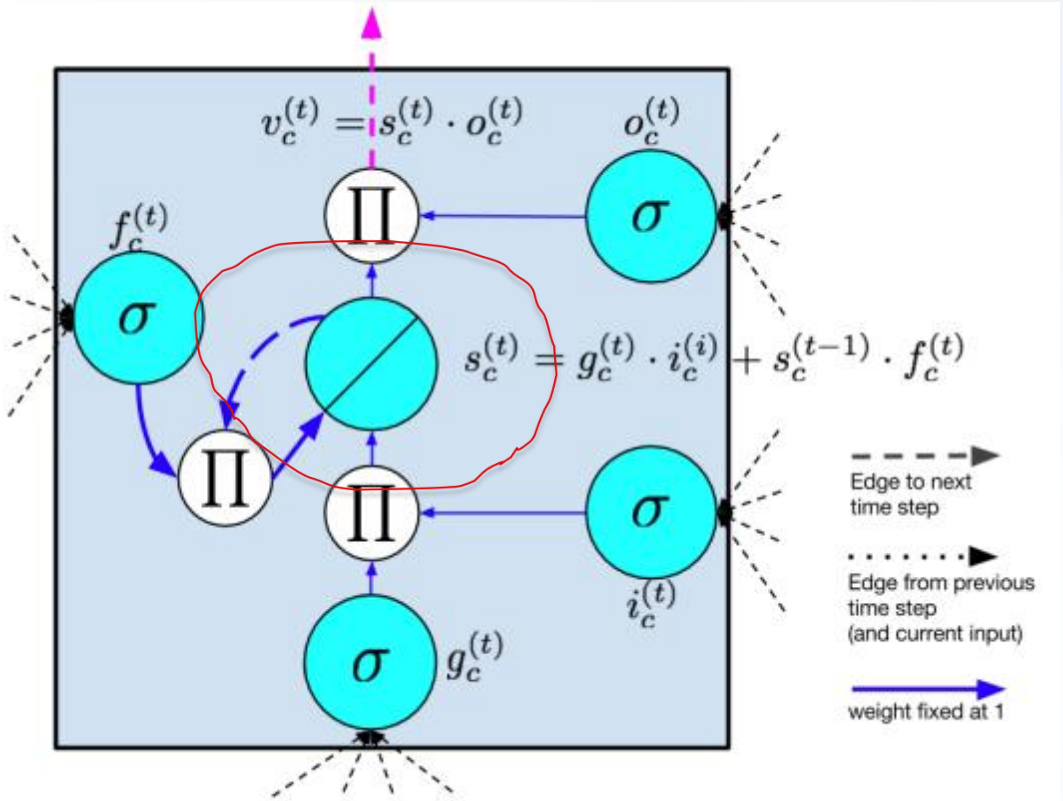
Solution: LSTM

LSTM Memory Cell



Differentiable and continuous analogy to digital memory blocks

Constant Error carousel



$$e_j(t) = f_j'\left(net_j(t)\right) e_j(t+1)\omega_{jj}$$

$$f_j'\left(net_j(t)\right)\omega_{jj} = 1.0$$

$$f_j\left(net_j(t)\right) = net_j(t)/\omega_{jj}$$

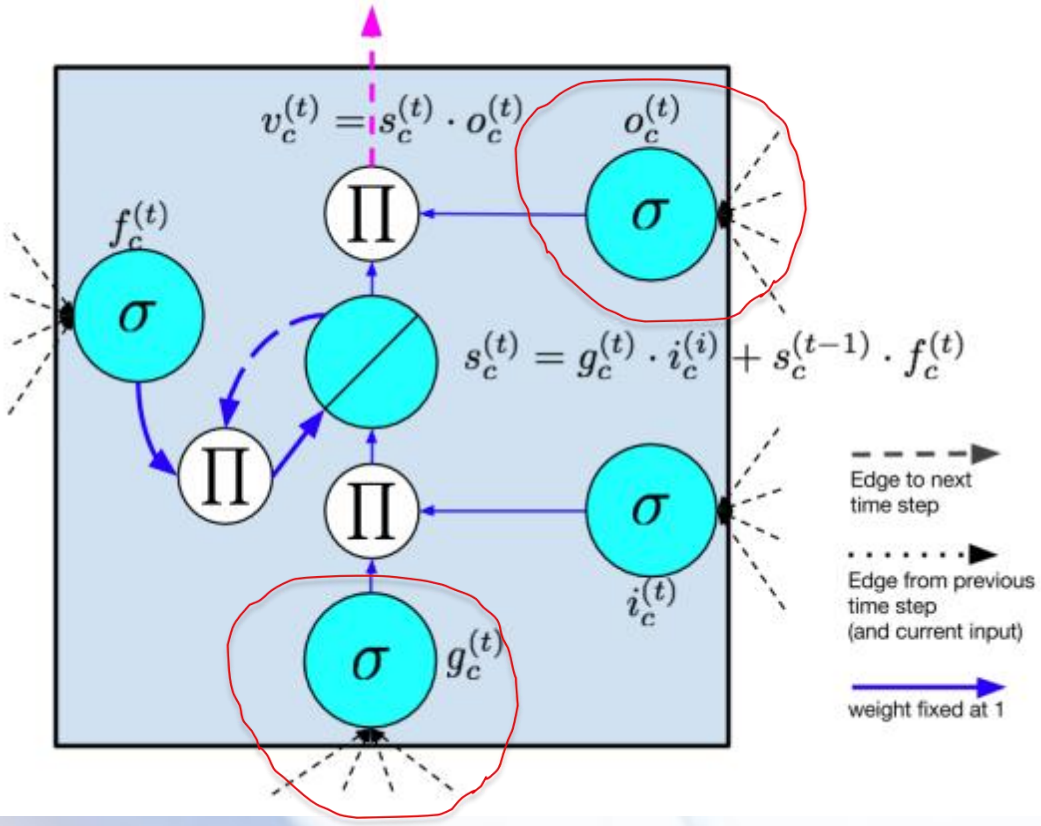
$$y_j(t+1) = f_j\left(net_j(t+1)\right)$$

$$= f_j\left(\omega_{jj} y_j(t)\right) = y_j(t)$$

$$f_j(x) = x \qquad \omega_{jj} = 1$$

Allow the network to propagate errors without modification

Input and output gate



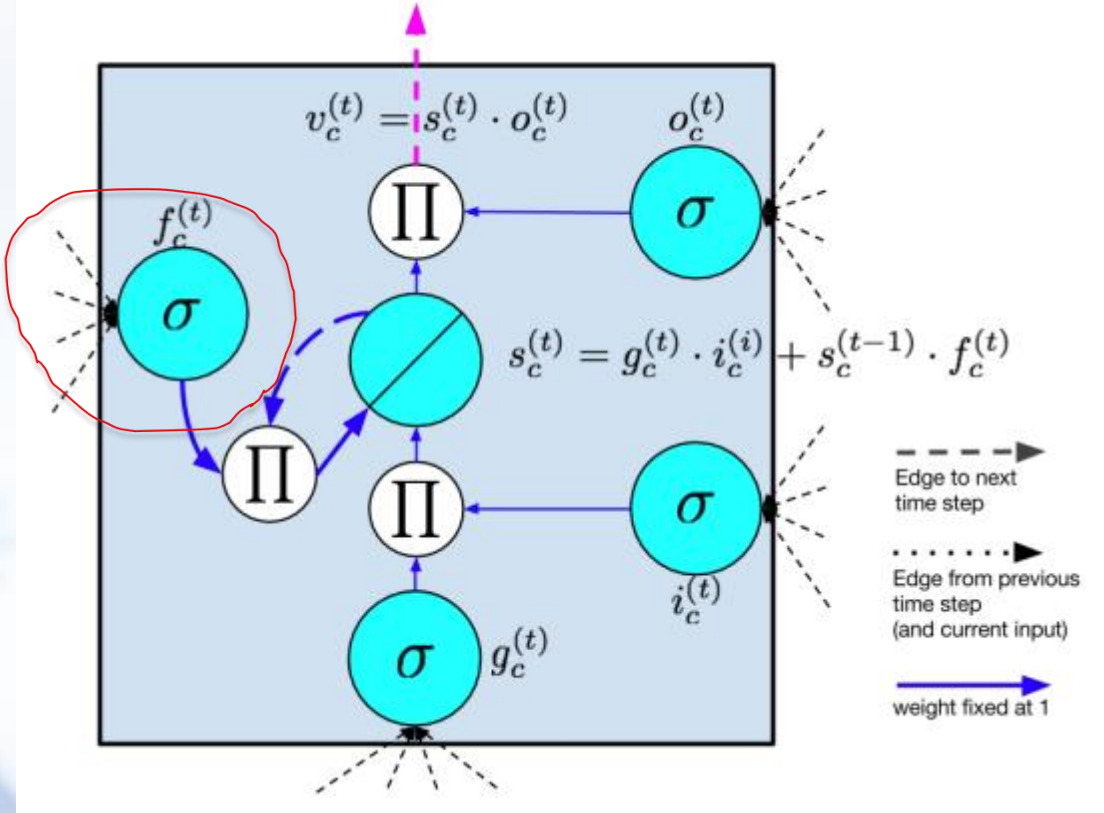
Output gate:

- Access information in memory
- Block irrelevant information

Input gate:

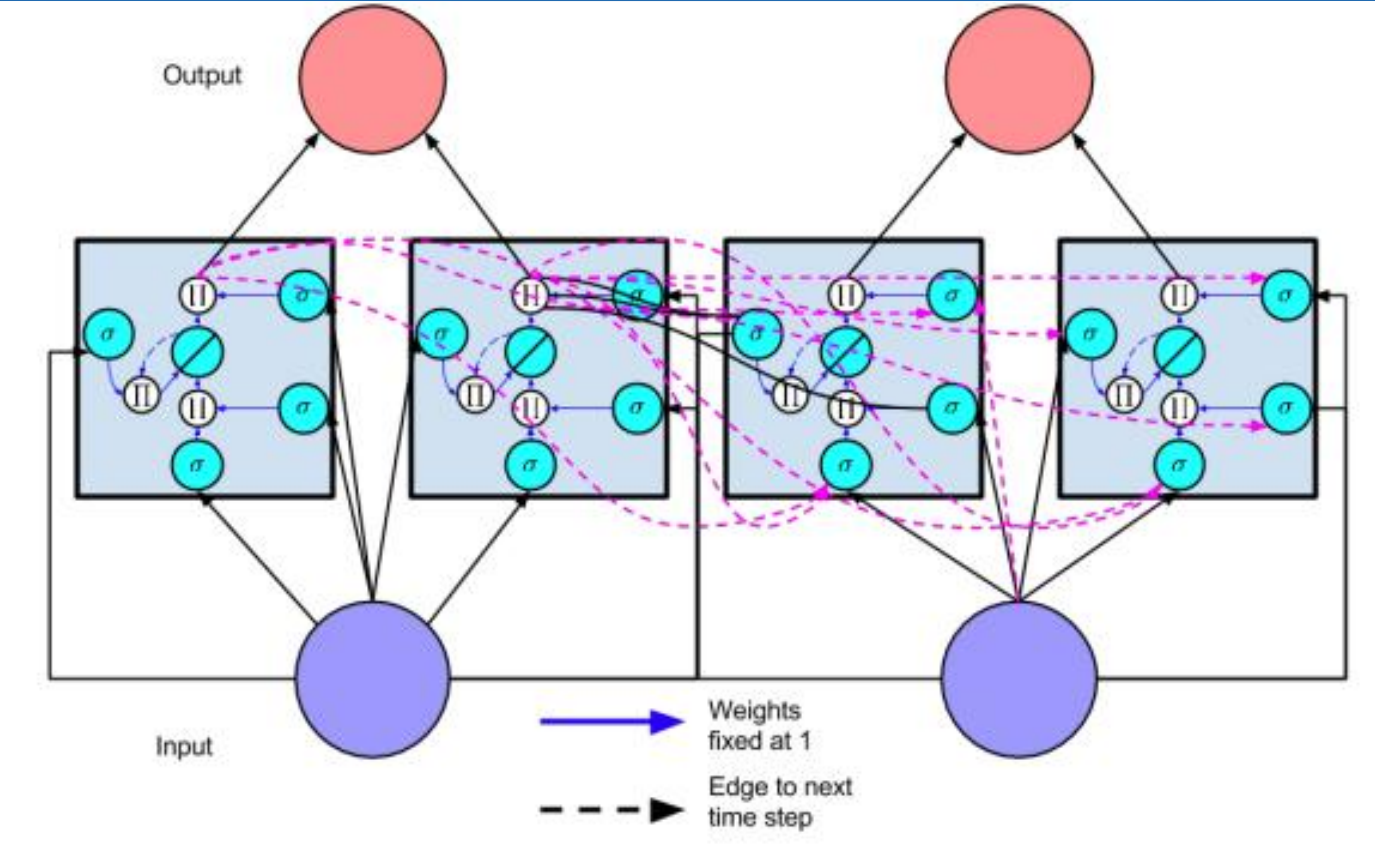
- Store input into memory
- Protect memory from overwritten by other irrelevant inputs

Forget gate



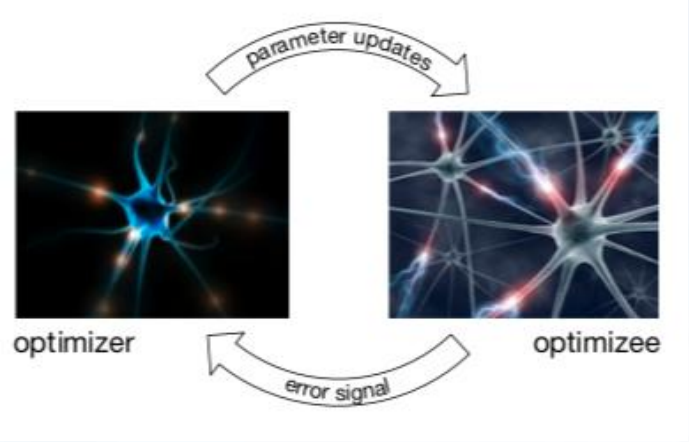
- Manipulates how much of previous cell state should be kept within the update to next state

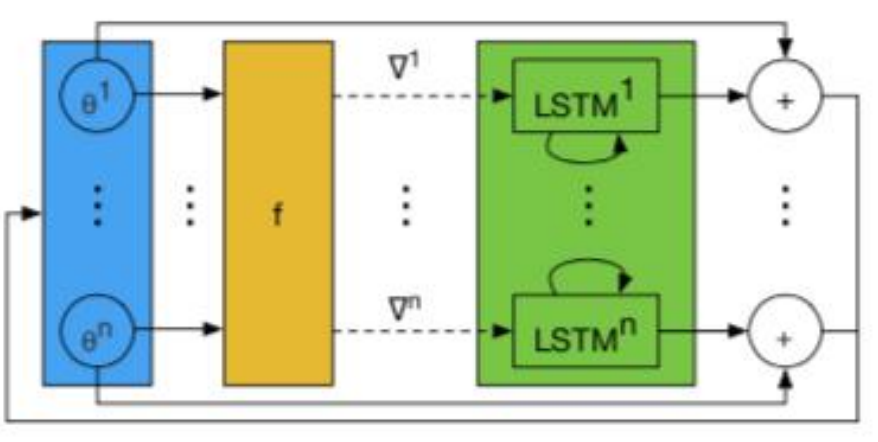
LSTM Networks

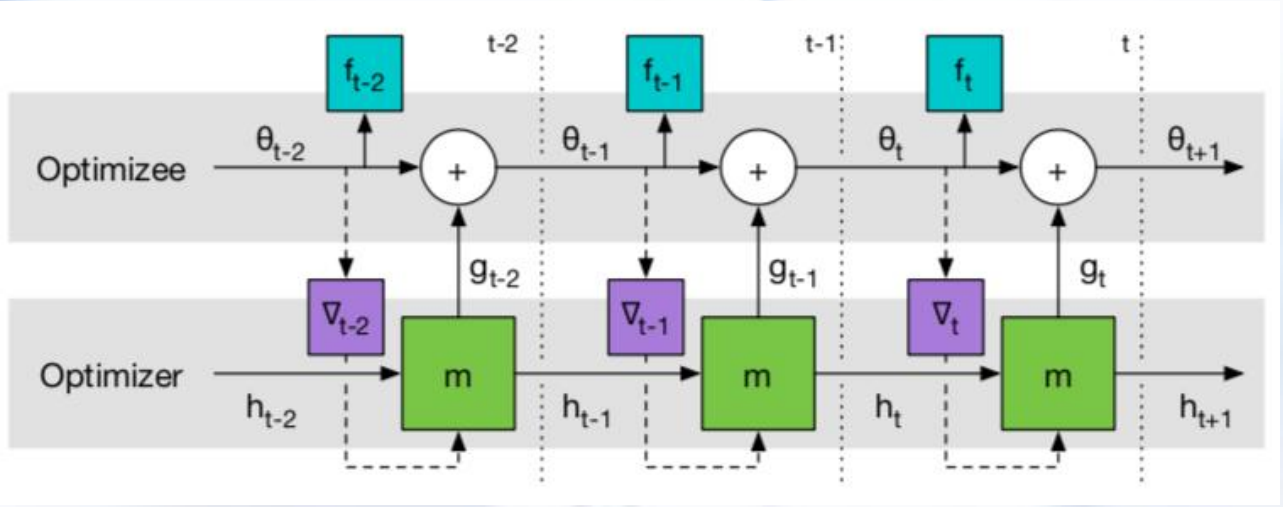


Use LSTM memory cell as neurons to construct the network, e.g., replacing hidden layer neurons in MLP and establishing the recurrent relationship.

LSTM Application: learning to learn







$$\theta_{t+1} = \theta_t + g_t\,,$$

$$\begin{bmatrix} g_t \\ h_{t+1} \end{bmatrix} = m(\nabla_t, h_t, \phi)$$

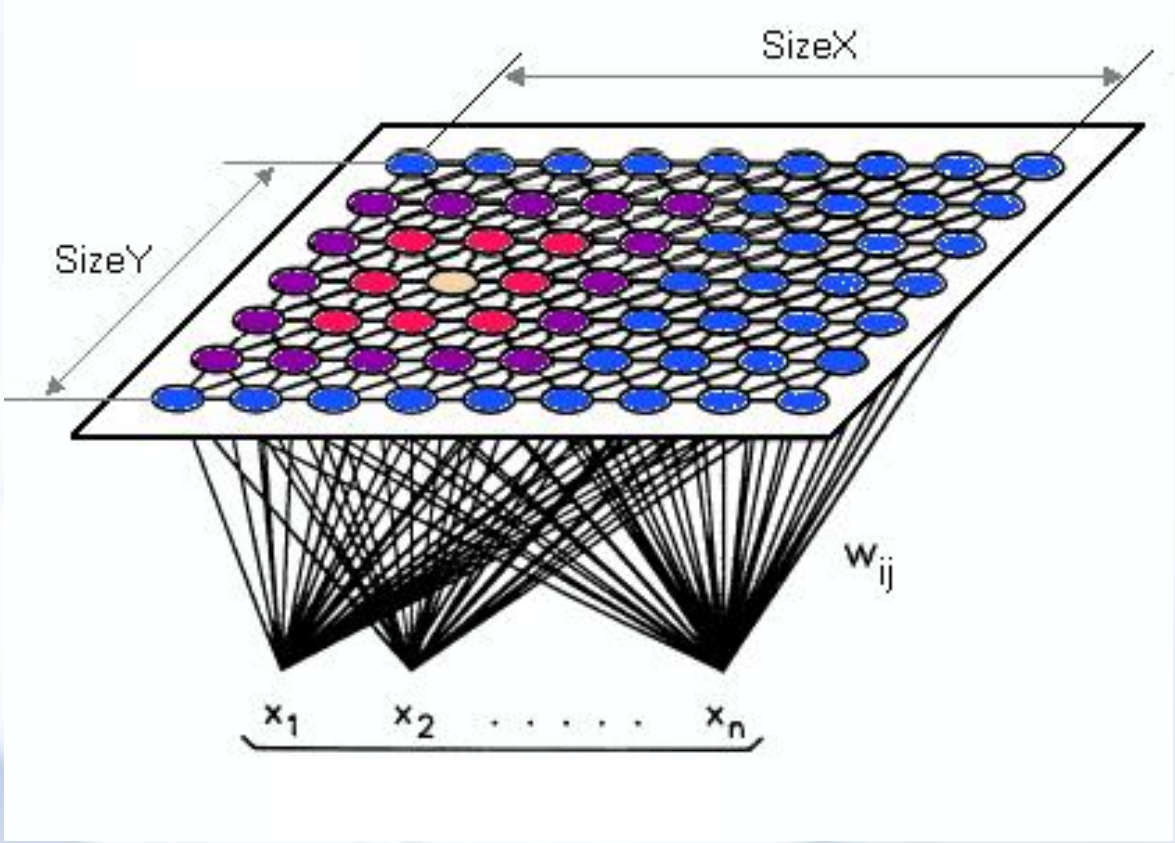
Andrychowicz et al., NIPS 2016

Part Ⅳ: Self-Organizing Feature Map (SOFM) of Kohonen

What is SOFM?

- Neural Network with Unsupervised Learning
- Dimensionality reduction concomitant with preservation of topological information.
- Three principals
  - Self-reinforcing
  - Competition
  - Cooperation

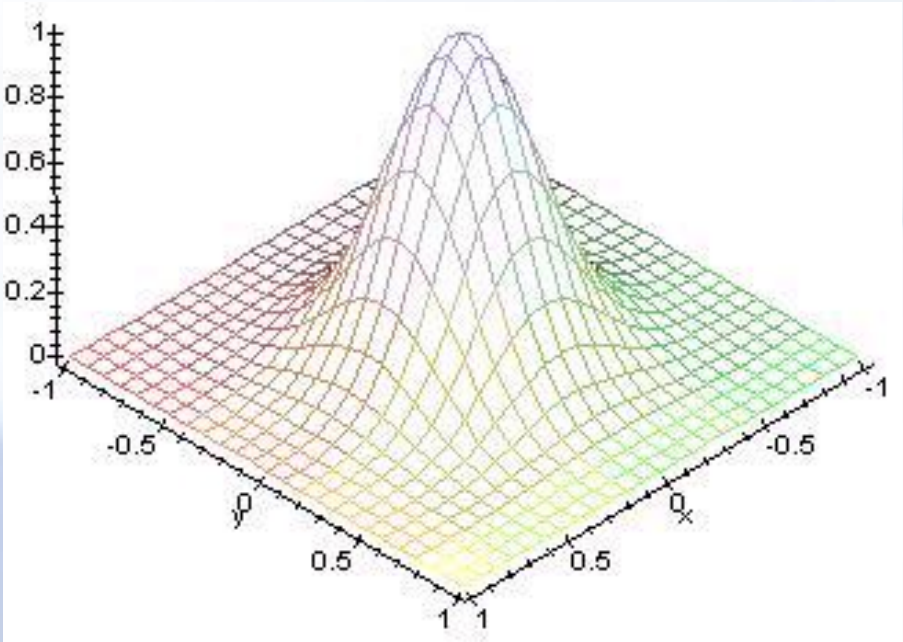
Structure of SOFM



Picture：

http://www.lohninger.com/helpcsuite/kohonen_network_-_background_information.htm

Mexican Hat Connectivity Pattern





Competition and Cooperation

Competition

- Finding the best matching weight vector for the present input.
- Criterion for determining the winning neuron:

  Maximum Inner Product → Minimum Euclidean Distance

Cooperation

- Identify a neighborhood around the winning neuron.
- Topological neighborhood can be of different shapes such as Square, Hexagonal, or Gaussian.
- The width of the neighborhood is a function of time: as epochs of training elapse, the neighborhood shrinks.

Adaptation

- Weights of neurons within the winning cluster are updated.
- Hebbian hypothesis but with a forgetting term scaled by some functions of the neuronal signal.

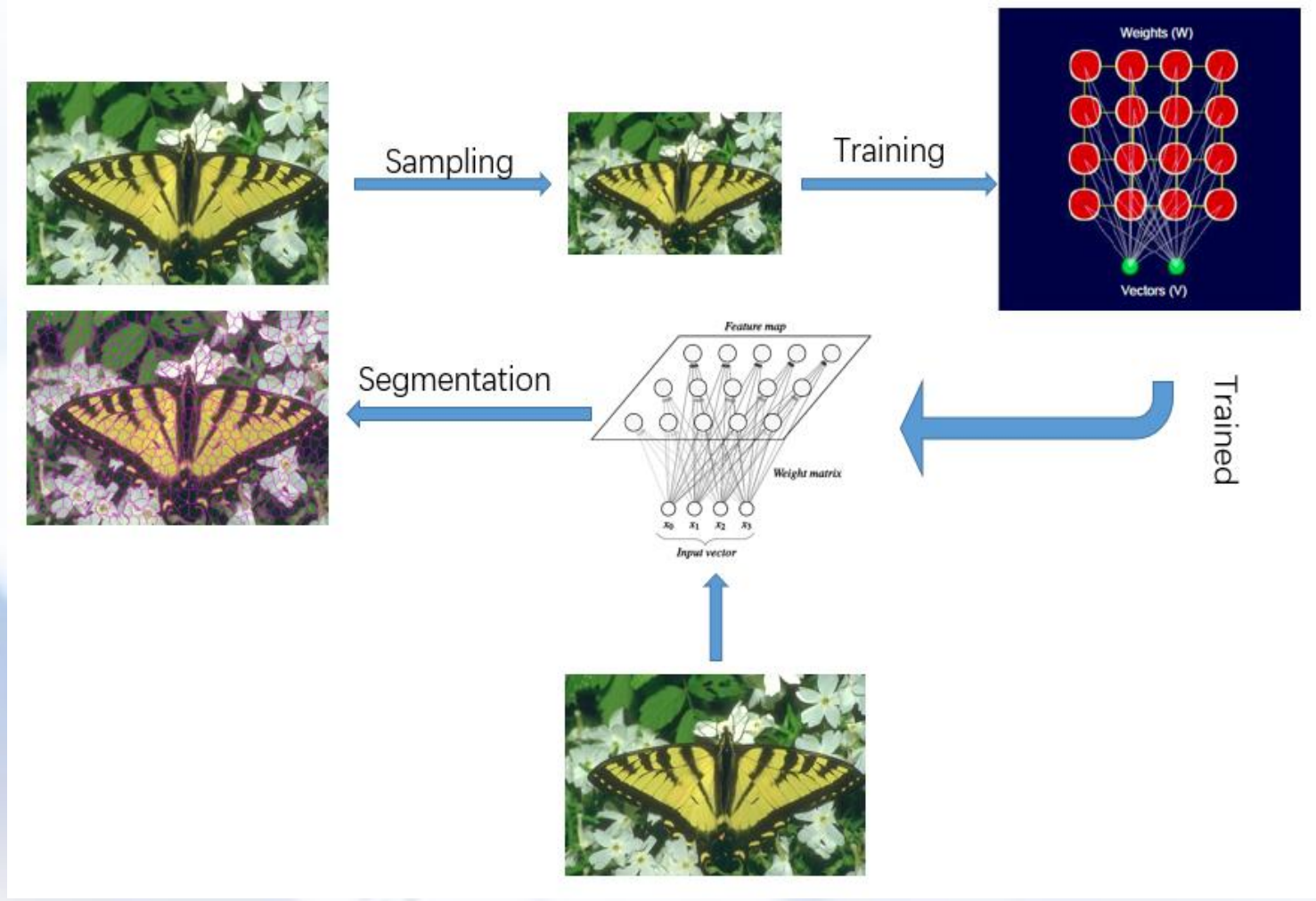
$$\boldsymbol{w}_j(t+1) = \begin{cases} \boldsymbol{w}_j(t) + \eta(t)\left(\boldsymbol{x}(t) - \boldsymbol{w}_j(t)\right), & j \in \Lambda_i(t) \\ \boldsymbol{w}_j(t), & j \notin \Lambda_i(t) \end{cases}$$

Operational Summary of SOFM algorithm

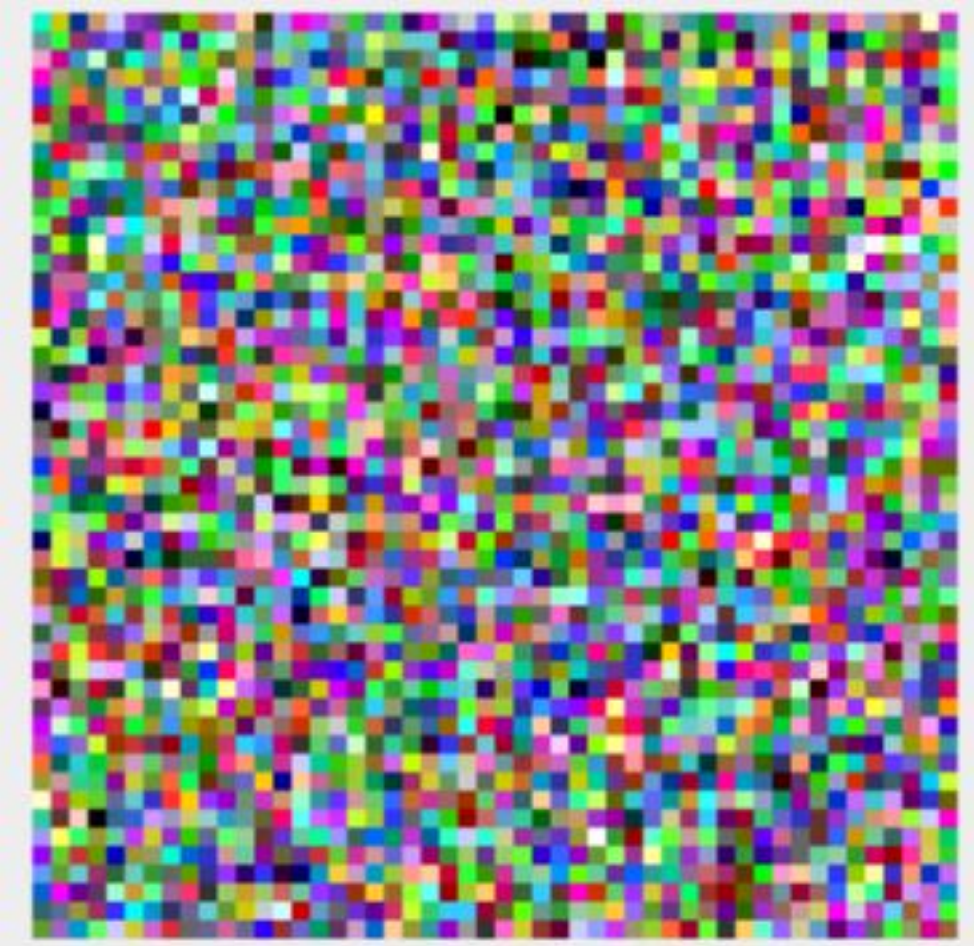
Repeat
{
Selection: Pick a sample
Similarity Matching: Find the winning neuron
Adaptation: Update synaptic vectors of ONLY the winning cluster.
Update: Update the learning rate and neighborhood

}Until (there is no observable change in the map)

SOFM Application: Superpixel Segmentation



SOFM Application: Color Clustering



Can you organize these colors in order, i.e., having similar colors close to each?

BIT.MLMR

End of Lecture 3

Can you understand artificial neural network and apply it to special applications?

LIUXIABI@BIT.EDU.CN





